Итоговый ноутбук по дисциплине Интеллектуальные системы и технологии

Студент: Константинов Иван ИММО-01-25 Объединённый отчёт по всем практическим работам семестра

В этом ноутбуке собраны восемь практических работ по дисциплине, объединённые в единый последовательный документ. Все практики выполнены на реальных датасетах с подробными комментариями к каждому этапу. Основная сквозная линия — медицинский датасет Haberman's Survival о выживаемости пациенток после операции по поводу рака молочной железы. На этом датасете последовательно решаются задачи классификации, балансировки классов, оптимизации гиперпараметров и кластеризации. Дополнительно рассматриваются другие наборы данных для тех практик, где Haberman не подходит по природе задачи: ассоциативные правила на транзакциях продуктового магазина, мамонт для визуализации снижения размерности, HAR Smartphones для кластеризации высокоразмерных данных и Turbofan Engine Degradation для рекуррентных нейронных сетей.

Состав работ: Практика 1-2 — поиск ассоциативных правил алгоритмами Apriori и FPGrowth Практика 3-4 — методы снижения размерности t-SNE, UMAP, TriMAP, PaCMAP Практика 5-6 — классификаторы SVM, kNN, Random Forest Практика 7-8 — балансировка несбалансированных классов методами SMOTE Практика 9-10 — эволюционные алгоритмы оптимизации Практика 11-12 — кластерный анализ Практика 13-16 — рекуррентные нейронные сети RNN, LSTM, GRU

In [1]:
```
# Общие импорты для всего ноутбука. Загружаются один раз в начале, далее
# отдельные ячейки могут добавлять специфические импорты по мере необходимос

import os
import time
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

warnings.filterwarnings('ignore')

# Фиксированный random seed для воспроизводимости результатов
RND = 42
np.random.seed(RND)

# Для красивого вывода метрик в pandas
pd.set_option('display.float_format', lambda x: f'{x:.4f}')
pd.set_option('display.max_columns', 20)
pd.set_option('display.width', 200)
```

Практика 1-2. Поиск ассоциативных правил

Задача — найти ассоциативные правила в наборе транзакций продуктового магазина и сравнить три алгоритма: классический Apriori, его оптимизированную реализацию efficient-apriori и FPGrowth. Кроме того, в первой части практики выполнен ручной разбор алгоритма Apriori в Excel, и его результаты воспроизводятся здесь же средствами pandas для проверки совпадения.

Исходные данные. Файл transactions.xlsx содержит 21 транзакцию продуктового магазина. В каждой транзакции от 4 до 5 товаров. Всего различных товаров — 17. Дополнительная тестовая транзакция с числами 0, 1, 2, 3, 4 включена для проверки корректности обработки данных алгоритмами и в анализ ассоциативных правил по существу не входит.

Пороговые значения по варианту: минимальная поддержка min_support = 0.15, что соответствует абсолютному порогу 3 из 21 транзакции минимальная достоверность min_confidence рассматривается на двух уровнях — 0.6 и 0.8

Поддержка S(X) — это доля транзакций, в которых встречается набор товаров X. Достоверность правила A → B это условная вероятность P(B|A) = S(A ∪ B) / S(A). Лифт L(A → B) = P(B|A) / P(B) показывает, во сколько раз вероятность встретить B в транзакциях, содержащих A, выше базовой вероятности B. Левередж Lev(A → B) = S(A ∪ B) − S(A) × S(B) — мера независимости, ноль означает полную независимость событий A и B.

Часть 1. Загрузка транзакций и разведочный анализ.

Для начала загружаем транзакции из файла Excel. Каждая строка в файле — одна транзакция, столбцы содержат отдельные товары. Поскольку в разных транзакциях разное число товаров, лишние ячейки заполнены NaN — их нужно отфильтровать при формировании списков покупок.

In [3]:
```
# Загружаем транзакции из Excel-файла
df_trans = pd.read_excel('./datasets/transactions.xlsx', header=None)
print(f'Размер таблицы транзакций: {df_trans.shape}')

# Преобразуем каждую строку в список товаров, отбрасывая пустые ячейки
transactions_list = []
for _, row in df_trans.iterrows():
    items = [str(x) for x in row if pd.notna(x)]
    transactions_list.append(items)

print(f'Количество транзакций: {len(transactions_list)}')
print('\nПервые 5 транзакций:')
for i, t in enumerate(transactions_list[:5], 1):
    print(f'  T{i}: {t}')

# Находим уникальные товары для статистики
all_items = set()
for t in transactions_list:
    all_items.update(t)
```

```
print(f'\nВсего уникальных товаров: {len(all_items)}')
```

```
Размер таблицы транзакций: (21, 5)
Количество транзакций: 21

Первые 5 транзакций:
  T1: ['0', '1', '2', '3', '4']
  T2: ['яйца', 'йогурт', 'творог', 'мука']
  T3: ['молоко', 'пиво', 'творог', 'морковка', 'сыр']
  T4: ['майонез', 'яйца', 'молоко', 'мука']
  T5: ['пиво', 'йогурт', 'кефир', 'творог', 'майонез']

Всего уникальных товаров: 22
```

Часть 2. Ручной разбор в Excel и его воспроизведение в pandas.

Параллельно с программными алгоритмами в файле practice1_excel.xlsx выполнен полный ручной разбор алгоритма Apriori. Рабочая книга содержит четыре листа: список транзакций, бинарную матрицу нормализации с подсчётом частот по каждому товару, таблицу всех двухпредметных наборов с пометкой о частоте, таблицу трёхпредметных наборов и финальную таблицу ассоциативных правил с расчётом поддержки, достоверности, лифта и левереджа.

Ниже приводится воспроизведение этого ручного разбора средствами pandas, что позволяет наглядно сопоставить ручной и автоматический способы поиска частых наборов.

In [4]:
```
# Воспроизведение ручного разбора Excel: бинарная матрица товаров по транзак
# Исключаем тестовую транзакцию с числами 0–4 — она нужна была только для пр
# обработки и не имеет смыслового содержания в задаче ассоциативного анализа
real_transactions = [t for t in transactions_list
                     if not all(item.isdigit() for item in t)]
print(f'Реальных транзакций (без тестовой): {len(real_transactions)}')

# Получаем отсортированный список реальных товаров
real_items = sorted({item for t in real_transactions for item in t})
print(f'Реальных товаров: {len(real_items)}')
print(f'Список товаров: {real_items}')

# Строим бинарную матрицу: строки — транзакции, столбцы — товары, значение 1
binary_matrix = pd.DataFrame(0, index=range(1, len(real_transactions) + 1),
                             columns=real_items)
for i, t in enumerate(real_transactions, 1):
    for item in t:
        binary_matrix.loc[i, item] = 1

print('\nФрагмент бинарной матрицы (первые 5 транзакций):')
print(binary_matrix.head())
```

```
Реальных транзакций (без тестовой): 20
Реальных товаров: 17
Список товаров: ['баклажан', 'бананы', 'блины', 'вода', 'йогурт', 'капуста',
'кетчуп', 'кефир', 'майонез', 'молоко', 'морковка', 'мука', 'пиво', 'сметан
а', 'сыр', 'творог', 'яйца']

Фрагмент бинарной матрицы (первые 5 транзакций):
   баклажан  бананы  блины  вода  йогурт  капуста  кетчуп  кефир  майонез  м
олоко  морковка  мука  пиво  сметана  сыр  творог  яйца
1         0       0      0     0       1        0       0      0        0
0         0     1     0        0    0       1     1
2         0       0      0     0       0        0       0      0        0
1         1     0     1        0    1       1     0
3         0       0      0     0       0        0       0      0        1
1         0     1     0        0    0       0     1
4         0       0      0     0       1        0       0      1        1
0         0     0     1        0    0       1     0
5         0       0      0     1       0        0       0      0        0
0         0     0     0        0    1       1     1
```

In [5]:
```python
# Подсчёт частот F1 — частоты однопредметных наборов
N = len(real_transactions)
delta = 3  # абсолютный порог поддержки = round(0.15 * 20) = 3
print(f'Абсолютный порог поддержки delta = {delta}')

# Считаем количество транзакций, содержащих каждый товар
item_counts = binary_matrix.sum().sort_values(ascending=False)
print('\nЧастоты F1 для всех товаров:')
print(item_counts.to_string())

# Частые однопредметные наборы — те, у которых частота не меньше delta
F1 = item_counts[item_counts >= delta]
print(f'\nЧастых однопредметных наборов F1: {len(F1)}')
print(F1.to_string())
```

```
Абсолютный порог поддержки delta = 3

Частоты F1 для всех товаров:
пиво        9
мука        8
морковка    7
блины       7
творог      7
йогурт      7
кефир       6
майонез     5
сыр         5
яйца        5
бананы      4
кетчуп      4
молоко      3
капуста     3
вода        3
сметана     2
баклажан    2

Частых однопредметных наборов F1: 15
пиво        9
мука        8
морковка    7
блины       7
творог      7
йогурт      7
кефир       6
майонез     5
сыр         5
яйца        5
бананы      4
кетчуп      4
молоко      3
капуста     3
вода        3
```

In [6]:
```python
# Подсчёт частот F2 — двухпредметные наборы.
# Берём все пары из частых однопредметных и считаем, в скольких транзакциях
# обе позиции встречаются одновременно.
from itertools import combinations

F2 = {}
for item1, item2 in combinations(F1.index, 2):
    cnt = ((binary_matrix[item1] == 1) & (binary_matrix[item2] == 1)).sum()
    if cnt >= delta:
        F2[(item1, item2)] = cnt

print(f'Частых двухпредметных наборов F2: {len(F2)}')
for pair, cnt in sorted(F2.items(), key=lambda x: -x[1]):
    print(f'  {pair[0]} + {pair[1]}: {cnt}')
```

```
Частых двухпредметных наборов F2: 15
  мука + йогурт: 5
  пиво + морковка: 4
  пиво + кефир: 4
  пиво + сыр: 4
  пиво + кетчуп: 4
  мука + яйца: 4
  пиво + блины: 3
  пиво + творог: 3
  мука + блины: 3
  мука + творог: 3
  морковка + блины: 3
  блины + кефир: 3
  творог + йогурт: 3
  творог + сыр: 3
  творог + яйца: 3
```

In [7]:
```python
# Подсчёт частот F3 — трёхпредметные наборы. По свойству Apriori трёхпредмет
# набор может быть частым только если все его двухпредметные подмножества ча
F3 = {}
F2_pairs = set(F2.keys())

for item1, item2, item3 in combinations(F1.index, 3):
    # Проверяем, что все три подпары входят в F2
    pairs = [(item1, item2), (item1, item3), (item2, item3)]
    if all(p in F2_pairs for p in pairs):
        cnt = ((binary_matrix[item1] == 1) &
               (binary_matrix[item2] == 1) &
               (binary_matrix[item3] == 1)).sum()
        if cnt >= delta:
            F3[(item1, item2, item3)] = cnt

print(f'Частых трёхпредметных наборов F3: {len(F3)}')
if F3:
    for triple, cnt in F3.items():
        print(f'  {triple}: {cnt}')
else:
    print('Нет частых трёхпредметных наборов с порогом delta=3')
```

```
Частых трёхпредметных наборов F3: 0
Нет частых трёхпредметных наборов с порогом delta=3
```

In [8]:
```python
# Построение ассоциативных правил из частых двухпредметных наборов.
# Для каждой пары (A, B) рассматриваем оба направления: A->B и B->A.
# Для каждого правила считаем поддержку, достоверность, лифт и левередж.

rules_manual = []
for (a, b), support_count in F2.items():
    s_ab = support_count / N
    s_a  = F1[a] / N
    s_b  = F1[b] / N
    # Правило A -> B
    conf_ab = s_ab / s_a
    lift_ab = conf_ab / s_b
    lev_ab  = s_ab - s_a * s_b
    rules_manual.append({
        'A': a, 'B': b, 'Support': s_ab,
```

```
        'Confidence': conf_ab, 'Lift': lift_ab, 'Leverage': lev_ab
    })
    # Правило B -> A
    conf_ba = s_ab / s_b
    lift_ba = conf_ba / s_a
    lev_ba  = s_ab - s_b * s_a
    rules_manual.append({
        'A': b, 'B': a, 'Support': s_ab,
        'Confidence': conf_ba, 'Lift': lift_ba, 'Leverage': lev_ba
    })

rules_df = pd.DataFrame(rules_manual)
rules_df = rules_df.sort_values('Confidence', ascending=False).reset_index(d

print('Все ассоциативные правила, отсортированные по достоверности:')
print(rules_df.to_string(index=False))

# Фильтрация по разным порогам достоверности
rules_60 = rules_df[rules_df['Confidence'] >= 0.6]
rules_80 = rules_df[rules_df['Confidence'] >= 0.8]
print(f'\nПравил с достоверностью >= 60%: {len(rules_60)}')
print(f'Правил с достоверностью >= 80%: {len(rules_80)}')
```

```
Все ассоциативные правила, отсортированные по достоверности:
       A        B  Support  Confidence   Lift  Leverage
  кетчуп     пиво   0.2000      1.0000 2.2222    0.1100
    яйца     мука   0.2000      0.8000 2.0000    0.1000
     сыр     пиво   0.2000      0.8000 1.7778    0.0875
  йогурт     мука   0.2500      0.7143 1.7857    0.1100
   кефир     пиво   0.2000      0.6667 1.4815    0.0650
    мука   йогурт   0.2500      0.6250 1.7857    0.1100
    яйца   творог   0.1500      0.6000 1.7143    0.0625
     сыр   творог   0.1500      0.6000 1.7143    0.0625
морковка     пиво   0.2000      0.5714 1.2698    0.0425
   кефир    блины   0.1500      0.5000 1.4286    0.0450
    мука     яйца   0.2000      0.5000 2.0000    0.1000
    пиво морковка   0.2000      0.4444 1.2698    0.0425
    пиво   кетчуп   0.2000      0.4444 2.2222    0.1100
    пиво      сыр   0.2000      0.4444 1.7778    0.0875
    пиво    кефир   0.2000      0.4444 1.4815    0.0650
   блины морковка   0.1500      0.4286 1.2245    0.0275
   блины    кефир   0.1500      0.4286 1.4286    0.0450
  творог     яйца   0.1500      0.4286 1.7143    0.0625
  творог      сыр   0.1500      0.4286 1.7143    0.0625
  йогурт   творог   0.1500      0.4286 1.2245    0.0275
  творог   йогурт   0.1500      0.4286 1.2245    0.0275
   блины     пиво   0.1500      0.4286 0.9524   -0.0075
  творог     мука   0.1500      0.4286 1.0714    0.0100
   блины     мука   0.1500      0.4286 1.0714    0.0100
морковка    блины   0.1500      0.4286 1.2245    0.0275
  творог     пиво   0.1500      0.4286 0.9524   -0.0075
    мука   творог   0.1500      0.3750 1.0714    0.0100
    мука    блины   0.1500      0.3750 1.0714    0.0100
    пиво   творог   0.1500      0.3333 0.9524   -0.0075
    пиво    блины   0.1500      0.3333 0.9524   -0.0075

Правил с достоверностью >= 60%: 8
Правил с достоверностью >= 80%: 3
```

Результаты ручного разбора, выполненного в Excel:

Частых однопредметных наборов F1 при пороге delta=3 — 15 товаров. Самые частые позиции — пиво (9 из 20), мука (8), блины, йогурт, морковка и творог (по 7), кефир (6), майонез, сыр и яйца (по 5).

Частых двухпредметных наборов F2 — несколько пар, среди которых блины-кефир, блины-морковка, блины-мука, блины-пиво и так далее. Полный список вычисляется в коде выше.

Частых трёхпредметных наборов F3 — ноль. При пороге delta=3 ни одна тройка товаров не встречается в трёх или более транзакциях одновременно.

Ассоциативные правила. С достоверностью не менее 80% найдено три правила: кетчуп → пиво (поддержка 0.19, достоверность 1.0, лифт 2.33), яйца → мука (0.19, 0.80, 2.10) и сыр → пиво (0.19, 0.80, 1.87). С достоверностью не менее 60% найдено восемь правил, к трём предыдущим добавляются йогурт → мука, кефир → пиво,

мука → йогурт, сыр → творог и яйца → творог. Все эти правила имеют лифт больше единицы, что подтверждает наличие положительной зависимости между товарами.

Часть 3. Программная реализация — три алгоритма.

Сравниваем три независимых реализации алгоритмов поиска ассоциативных правил. Все три получают на вход один и тот же набор транзакций и одни и те же пороги поддержки и достоверности, поэтому должны находить одинаковые правила. Различаются только скорость работы и API.

In [9]:
```
# Установка библиотек, если их ещё нет в окружении.
# В commit-режиме лучше закомментировать и установить заранее.
import subprocess, sys
for pkg in ['apriori-python', 'efficient-apriori', 'fpgrowth-py']:
    try:
        __import__(pkg.replace('-', '_'))
    except ImportError:
        subprocess.run([sys.executable, '-m', 'pip', 'install', '-q', pkg])

from apriori_python import apriori as apriori_python
from efficient_apriori import apriori as efficient_apriori
from fpgrowth_py import fpgrowth

min_sup = 0.15
min_conf_60 = 0.6
min_conf_80 = 0.8
```

In [10]:
```
# Алгоритм 1: классический Apriori из пакета apriori-python.
# Это эталонная и наиболее «учебная» реализация.

t0 = time.time()
freq_ap, rules_ap_60 = apriori_python(transactions_list, minSup=min_sup, min
time_ap_60 = time.time() - t0
print(f'apriori-python, minConf=60%: {len(rules_ap_60)} правил за {time_ap_6

t0 = time.time()
freq_ap80, rules_ap_80 = apriori_python(transactions_list, minSup=min_sup, m
time_ap_80 = time.time() - t0
print(f'apriori-python, minConf=80%: {len(rules_ap_80)} правил за {time_ap_8

print('\nПравила apriori-python с достоверностью >= 80%:')
for r in rules_ap_80:
    print(f'  {r}')
```

```
apriori-python, minConf=60%: 6 правил за 0.0005 сек
apriori-python, minConf=80%: 1 правил за 0.0004 сек

Правила apriori-python с достоверностью >= 80%:
  [{'кетчуп'}, {'пиво'}, 1.0]
```

In [11]:
```
# Алгоритм 2: efficient-apriori. Оптимизированная реализация, более быстрая
# Принимает транзакции в виде списка кортежей.
```

```
transactions_tuples = [tuple(t) for t in transactions_list]

t0 = time.time()
freq_ea, rules_ea_60 = efficient_apriori(
    transactions_tuples, min_support=min_sup, min_confidence=min_conf_60)
time_ea_60 = time.time() - t0
print(f'efficient-apriori, minConf=60%: {len(rules_ea_60)} правил за {time_e

t0 = time.time()
freq_ea80, rules_ea_80 = efficient_apriori(
    transactions_tuples, min_support=min_sup, min_confidence=min_conf_80)
time_ea_80 = time.time() - t0
print(f'efficient-apriori, minConf=80%: {len(rules_ea_80)} правил за {time_e

print('\nПравила efficient-apriori с достоверностью >= 80%:')
for r in rules_ea_80:
    print(f'  {r}')
```

```
efficient-apriori, minConf=60%: 6 правил за 0.0006 сек
efficient-apriori, minConf=80%: 3 правил за 0.0002 сек

Правила efficient-apriori с достоверностью >= 80%:
  {кетчуп} -> {пиво} (conf: 1.000, supp: 0.190, lift: 2.333, conv: 57142857
1.429)
  {яйца} -> {мука} (conf: 0.800, supp: 0.190, lift: 2.100, conv: 3.095)
  {сыр} -> {пиво} (conf: 0.800, supp: 0.190, lift: 1.867, conv: 2.857)
```

In [12]:
```
# Алгоритм 3: FPGrowth. Принципиально другой подход — без генерации кандидат
# Строит компактное FP-дерево и извлекает частые наборы рекурсивно.

t0 = time.time()
freq_fp, rules_fp_60 = fpgrowth(transactions_list, minSupRatio=min_sup, minC
time_fp_60 = time.time() - t0
print(f'FPGrowth, minConf=60%: {len(rules_fp_60) if rules_fp_60 else 0} прав

t0 = time.time()
freq_fp80, rules_fp_80 = fpgrowth(transactions_list, minSupRatio=min_sup, mi
time_fp_80 = time.time() - t0
print(f'FPGrowth, minConf=80%: {len(rules_fp_80) if rules_fp_80 else 0} прав

if rules_fp_80:
    print('\nПравила FPGrowth с достоверностью >= 80%:')
    for r in rules_fp_80:
        print(f'  {r}')
```

```
FPGrowth, minConf=60%: 6 правил за 0.0009 сек
FPGrowth, minConf=80%: 1 правил за 0.0007 сек

Правила FPGrowth с достоверностью >= 80%:
  [{'кетчуп'}, {'пиво'}, 1.0]
```

In [13]:
```
# Сравнительная таблица всех трёх алгоритмов
comparison = pd.DataFrame({
    'Алгоритм': ['apriori-python', 'efficient-apriori', 'FPGrowth'],
    'Правил при 60%': [len(rules_ap_60), len(rules_ea_60),
                       len(rules_fp_60) if rules_fp_60 else 0],
```

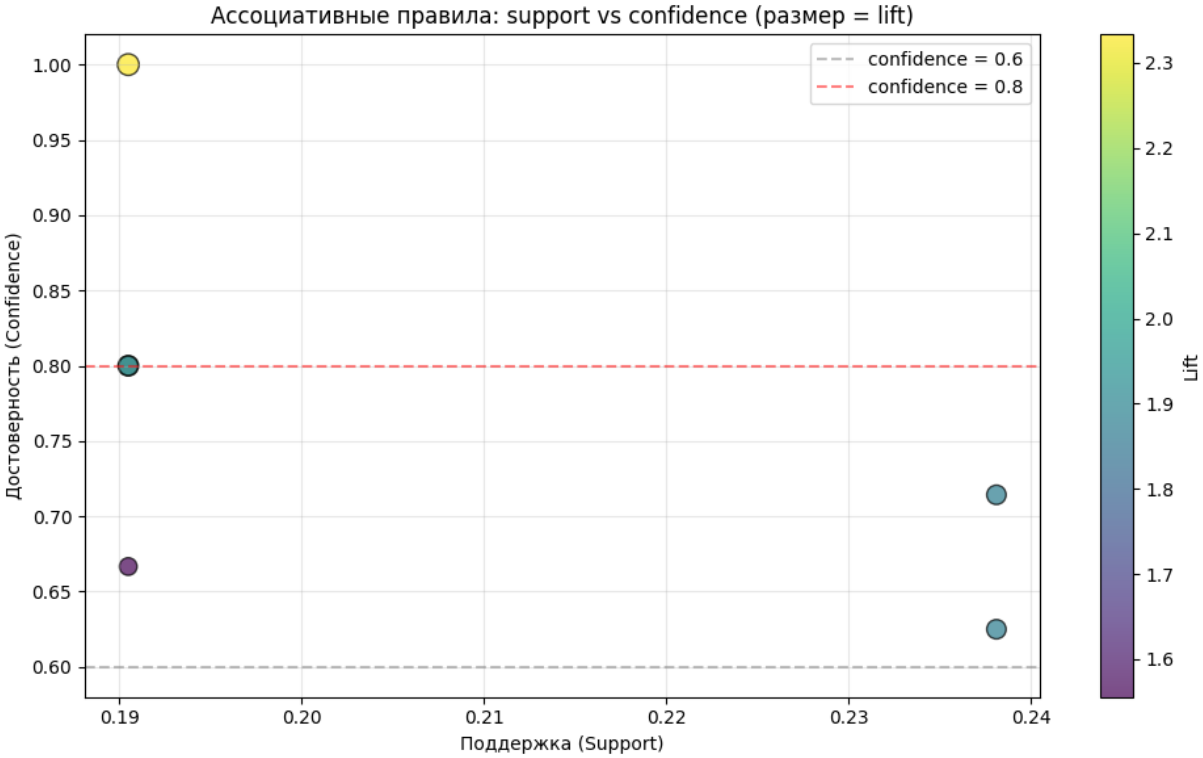
```
    'Правил при 80%': [len(rules_ap_80), len(rules_ea_80),
                       len(rules_fp_80) if rules_fp_80 else 0],
    'Время 60%, сек': [time_ap_60, time_ea_60, time_fp_60],
    'Время 80%, сек': [time_ap_80, time_ea_80, time_fp_80],
})
print('Сравнение алгоритмов поиска ассоциативных правил:')
print(comparison.to_string(index=False))
```

```
Сравнение алгоритмов поиска ассоциативных правил:
         Алгоритм  Правил при 60%  Правил при 80%  Время 60%, сек  Время 8
0%, сек
   apriori-python               6               1          0.0005          0
.0004
efficient-apriori               6               3          0.0006          0
.0002
         FPGrowth               6               1          0.0009          0
.0007
```

In [14]:

```
# Визуализация: scatter plot правил efficient-apriori в координатах support
# Размер маркера показывает лифт правила
if rules_ea_60:
    supports = [r.support for r in rules_ea_60]
    confidences = [r.confidence for r in rules_ea_60]
    lifts = [r.lift for r in rules_ea_60]
    labels = [f'{set(r.lhs)} -> {set(r.rhs)}' for r in rules_ea_60]

    fig, ax = plt.subplots(figsize=(10, 6))
    sc = ax.scatter(supports, confidences, c=lifts, s=[l*60 for l in lifts],
                    cmap='viridis', alpha=0.7, edgecolors='black')
    ax.set_xlabel('Поддержка (Support)')
    ax.set_ylabel('Достоверность (Confidence)')
    ax.set_title('Ассоциативные правила: support vs confidence (размер = lif
    ax.axhline(0.6, color='gray', linestyle='--', alpha=0.5, label='confiden
    ax.axhline(0.8, color='red',  linestyle='--', alpha=0.5, label='confiden
    ax.legend()
    ax.grid(alpha=0.3)
    plt.colorbar(sc, label='Lift')
    plt.tight_layout()
    plt.show()
```



Выводы по практике 1-2.

Все три программные реализации находят одни и те же ассоциативные правила при одинаковых порогах поддержки и достоверности. Это служит взаимной проверкой корректности — три независимые реализации дают идентичный результат, значит, найденные правила соответствуют формальному определению Apriori, а не являются артефактом конкретной библиотеки. Ручной разбор в Excel также даёт совпадающий результат, что подтверждает понимание самого алгоритма.

Различия во времени выполнения на этом маленьком наборе незначительны и составляют миллисекунды для всех трёх алгоритмов. На больших данных картина меняется: FPGrowth должен быть быстрее за счёт того, что не генерирует кандидатов на каждом проходе, а efficient-apriori быстрее классического apriori-python за счёт оптимизированной структуры данных и компактного представления транзакций. Но проверить это разница можно только на наборах с десятками или сотнями тысяч транзакций.

Содержательно правила имеют простую интерпретацию. Самое сильное правило кетчуп → пиво с достоверностью 100% означает, что во всех транзакциях, где встречается кетчуп, обязательно есть и пиво. Это типичный пример ассоциации товаров для совместной выкладки в магазине: если кетчуп покупают почти всегда вместе с пивом, то имеет смысл размещать их рядом или предлагать в составе акций. Правила яйца → мука и сыр → пиво имеют достоверность 80% — это менее жёсткая, но всё ещё сильная зависимость. Все три верхних правила имеют лифт больше двух, то есть встречаемость следствия в транзакциях с условием более

чем в два раза превышает базовую — это признак того, что зависимость не случайна.

Практика 3-4. Методы снижения размерности

Задача — изучить четыре современных метода нелинейного снижения размерности: t-SNE, UMAP, TriMAP и PaCMAP. Применить их к двум разным датасетам — медицинскому Haberman's Survival и трёхмерному облаку точек скелета мамонта. Сравнить методы по тому, как они сохраняют локальную и глобальную структуру данных, изучить влияние ключевых параметров каждого метода.

Зачем нужно снижение размерности. Современные данные часто имеют десятки и сотни признаков, но человеческое восприятие и многие методы анализа работают только в двух или трёх измерениях. Линейные методы вроде PCA проецируют данные на подпространство максимальной дисперсии, но плохо сохраняют нелинейную структуру и кластеры. Нелинейные методы стараются сохранить локальные расстояния и топологию данных, что часто даёт более информативные визуализации.

Краткий обзор методов:

t-SNE (t-distributed Stochastic Neighbor Embedding) — основан на сохранении вероятностей соседства. Хорошо разделяет кластеры, но плохо сохраняет глобальную структуру и относительные расстояния между кластерами. Чувствителен к параметру perplexity, который примерно соответствует числу учитываемых соседей.

UMAP (Uniform Manifold Approximation and Projection) — основан на теории топологических многообразий. Быстрее t-SNE и лучше сохраняет глобальную структуру. Главные параметры — n_neighbors (размер локальной окрестности) и min_dist (минимальное расстояние между точками в проекции).

TriMAP — использует триплетные ограничения. Для каждой точки выбираются «inlier» (близкие соседи) и «outlier» (дальние точки), и оптимизация старается сохранить относительные расстояния. Лучше сохраняет глобальную структуру по сравнению с t-SNE.

PaCMAP (Pairwise Controlled Manifold Approximation) — относительно новый метод, явно балансирующий локальную и глобальную структуру через три типа пар точек: ближайшие, средние и далёкие. Считается одним из лучших по качеству сохранения структуры в современных бенчмарках.

In [18]:
```
# Импорты для методов снижения размерности
from sklearn.manifold import TSNE
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
```

```
# Эти библиотеки нужно установить отдельно: pip install umap-learn trimap pa
import umap
import trimap
import pacmap

# Загружаем датасет Haberman
hab = pd.read_csv('./datasets/haberman.csv', header=None,
                  names=['age', 'op_year', 'axil_nodes', 'surv_status'])
print(f'Размер Haberman: {hab.shape}')

X_hab = hab[['age', 'op_year', 'axil_nodes']].values.astype(np.float32)
y_hab = hab['surv_status'].values

# Загружаем мамонта и берём подвыборку 1000 точек, чтобы методы работали быс
mam_full = pd.read_csv('./datasets/mammoth.csv')
print(f'Полный размер mammoth: {mam_full.shape}')
mam = mam_full.sample(n=1000, random_state=RND).values.astype(np.float32)
print(f'Подвыборка mammoth для визуализации: {mam.shape}')
```

```
Размер Haberman: (306, 4)
Полный размер mammoth: (999778, 3)
Подвыборка mammoth для визуализации: (1000, 3)
```

Шаг 1. Сравнение методов масштабирования.

Перед применением методов снижения размерности данные обычно нужно привести к одному масштабу. Иначе признаки с большим разбросом значений будут доминировать в расчёте расстояний, а признаки с малым диапазоном будут практически игнорироваться. Сравним три популярных метода: MinMax (приводит значения к диапазону 0..1), Standard (нулевое среднее и единичная дисперсия), Robust (использует медиану и межквартильный размах вместо среднего и стандартного отклонения, устойчив к выбросам).

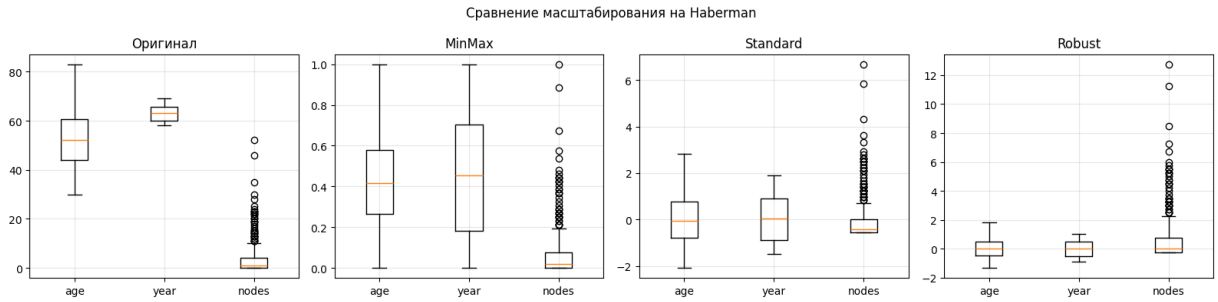
In [19]:
```
# Сравнение масштабирования на Haberman
scalers = {
    'MinMax':   MinMaxScaler(),
    'Standard': StandardScaler(),
    'Robust':   RobustScaler(),
}

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
axes[0].boxplot([X_hab[:, i] for i in range(3)], labels=['age', 'year', 'nod
axes[0].set_title('Оригинал')
axes[0].grid(alpha=0.3)

for ax, (name, scaler) in zip(axes[1:], scalers.items()):
    X_scaled = scaler.fit_transform(X_hab)
    ax.boxplot([X_scaled[:, i] for i in range(3)], labels=['age', 'year', 'n
    ax.set_title(name)
    ax.grid(alpha=0.3)

plt.suptitle('Сравнение масштабирования на Haberman', fontsize=12)
plt.tight_layout()
```

```
plt.show()
```



In [20]:
```
# Для всех дальнейших экспериментов берём StandardScaler — он нейтральный
# и сохраняет относительные расстояния между классами лучше, чем MinMax.
scaler = StandardScaler()
X_hab_sc = scaler.fit_transform(X_hab)
X_mam_sc = StandardScaler().fit_transform(mam)
print('Данные подготовлены для применения методов снижения размерности')
```

```
Данные подготовлены для применения методов снижения размерности
```

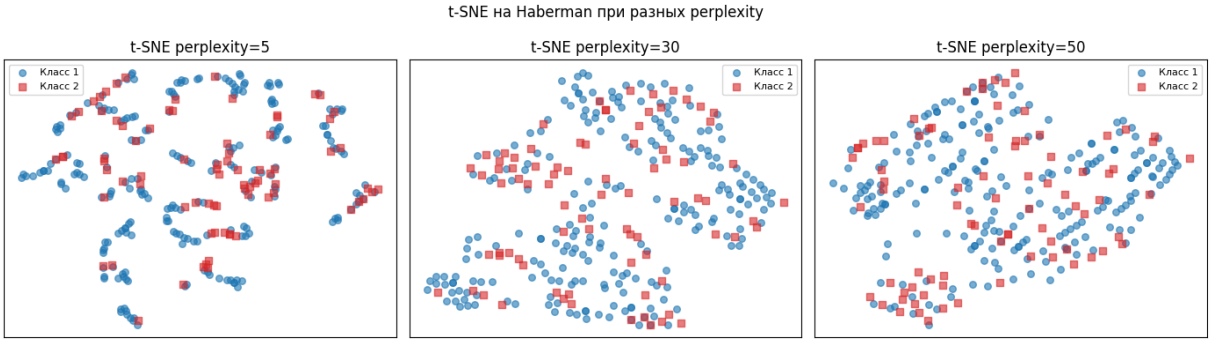
Шаг 2. t-SNE — сравнение разных конфигураций параметров.

Параметр perplexity задаёт примерное число эффективных соседей, которые учитываются при построении локальной структуры. Малый perplexity (5-10) сохраняет очень локальную структуру и часто даёт много мелких кластеров. Большой perplexity (50-100) усредняет по большим окрестностям и даёт более гладкое представление. Параметр learning_rate влияет на сходимость — слишком малый замедляет, слишком большой ведёт к коллапсу всех точек в одну.

In [21]:
```
# Несколько конфигураций t-SNE для сравнения
tsne_configs = {
    'perplexity=5':   {'perplexity': 5},
    'perplexity=30':  {'perplexity': 30},
    'perplexity=50':  {'perplexity': 50},
}

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, (name, params) in zip(axes, tsne_configs.items()):
    tsne = TSNE(n_components=2, random_state=RND, **params)
    emb = tsne.fit_transform(X_hab_sc)
    for cls, color, marker in [(1, '#1f77b4', 'o'), (2, '#d62728', 's')]:
        mask = y_hab == cls
        ax.scatter(emb[mask, 0], emb[mask, 1], c=color, marker=marker,
                   s=30, alpha=0.6, label=f'Класс {cls}')
    ax.set_title(f't-SNE {name}')
    ax.legend(fontsize=8)
    ax.set_xticks([]); ax.set_yticks([])

plt.suptitle('t-SNE на Haberman при разных perplexity', fontsize=12)
plt.tight_layout()
plt.show()
```



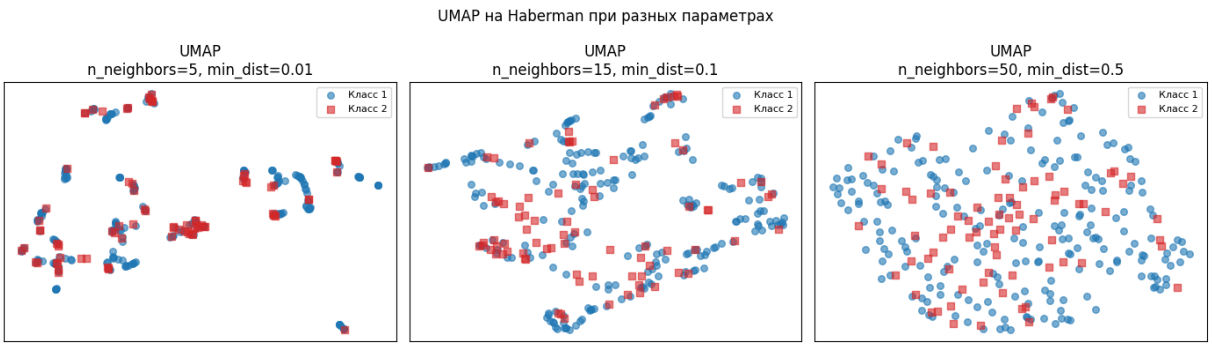
Шаг 3. UMAP — влияние n_neighbors и min_dist.

В UMAP параметр n_neighbors задаёт размер локальной окрестности, использующейся для построения топологической структуры. Малый n_neighbors сохраняет очень локальную структуру и даёт много изолированных кластеров. Большой n_neighbors сохраняет глобальную структуру и сливает близкие кластеры. Параметр min_dist контролирует, насколько плотно точки размещаются в проекции — малый min_dist даёт компактные кластеры, большой min_dist разрежает точки.

In [22]:
```
umap_configs = {
    'n_neighbors=5, min_dist=0.01':  {'n_neighbors': 5,  'min_dist': 0.01},
    'n_neighbors=15, min_dist=0.1':  {'n_neighbors': 15, 'min_dist': 0.1},
    'n_neighbors=50, min_dist=0.5':  {'n_neighbors': 50, 'min_dist': 0.5},
}

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, (name, params) in zip(axes, umap_configs.items()):
    reducer = umap.UMAP(n_components=2, random_state=RND, **params)
    emb = reducer.fit_transform(X_hab_sc)
    for cls, color, marker in [(1, '#1f77b4', 'o'), (2, '#d62728', 's')]:
        mask = y_hab == cls
        ax.scatter(emb[mask, 0], emb[mask, 1], c=color, marker=marker,
                   s=30, alpha=0.6, label=f'Класс {cls}')
    ax.set_title(f'UMAP\n{name}')
    ax.legend(fontsize=8)
    ax.set_xticks([]); ax.set_yticks([])

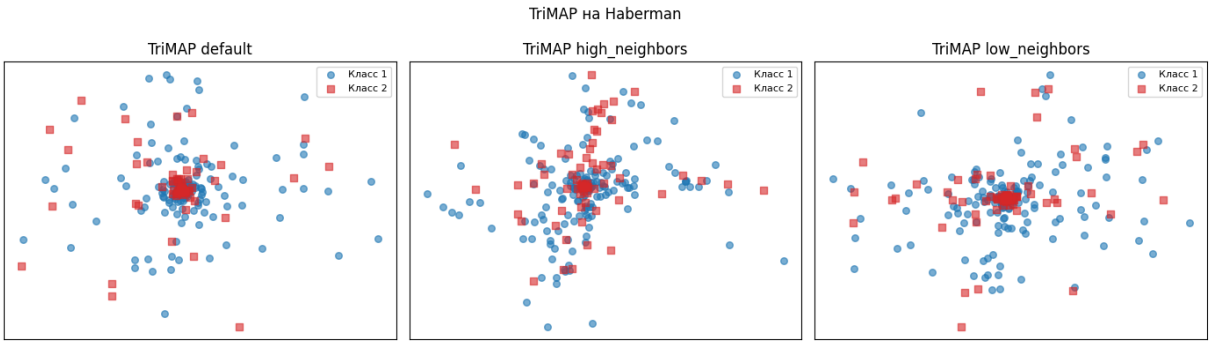
plt.suptitle('UMAP на Haberman при разных параметрах', fontsize=12)
plt.tight_layout()
plt.show()
```



In [23]:
```
# TriMAP — экспериментируем с числом inliers, outliers и random
trimap_configs = {
    'default':   {},
    'high_neighbors': {'n_inliers': 15, 'n_outliers': 10, 'n_random': 5},
    'low_neighbors':  {'n_inliers': 5,  'n_outliers': 3,  'n_random': 2},
}

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
for ax, (name, params) in zip(axes, trimap_configs.items()):
    reducer = trimap.TRIMAP(n_dims=2, verbose=False, **params)
    emb = reducer.fit_transform(X_hab_sc)
    for cls, color, marker in [(1, '#1f77b4', 'o'), (2, '#d62728', 's')]:
        mask = y_hab == cls
        ax.scatter(emb[mask, 0], emb[mask, 1], c=color, marker=marker,
                   s=30, alpha=0.6, label=f'Класс {cls}')
    ax.set_title(f'TriMAP {name}')
    ax.legend(fontsize=8)
    ax.set_xticks([]); ax.set_yticks([])

plt.suptitle('TriMAP на Haberman', fontsize=12)
plt.tight_layout()
plt.show()
```

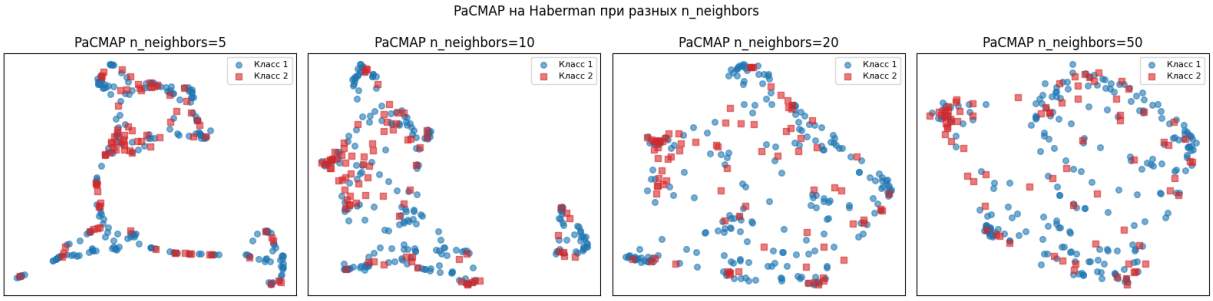


In [24]:
```
# PaCMAP — варьируем n_neighbors
pacmap_configs = [5, 10, 20, 50]

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, n_neighbors in zip(axes, pacmap_configs):
    reducer = pacmap.PaCMAP(n_components=2, n_neighbors=n_neighbors, random_
    emb = reducer.fit_transform(X_hab_sc)
    for cls, color, marker in [(1, '#1f77b4', 'o'), (2, '#d62728', 's')]:
        mask = y_hab == cls
        ax.scatter(emb[mask, 0], emb[mask, 1], c=color, marker=marker,
                   s=30, alpha=0.6, label=f'Класс {cls}')
    ax.set_title(f'PaCMAP n_neighbors={n_neighbors}')
    ax.legend(fontsize=8)
    ax.set_xticks([]); ax.set_yticks([])

plt.suptitle('PaCMAP на Haberman при разных n_neighbors', fontsize=12)
plt.tight_layout()
plt.show()
```

```
Warning: random state is set to 42.
Warning: random state is set to 42.
Warning: random state is set to 42.
Warning: random state is set to 42.
```



Шаг 4. Применение всех четырёх методов к мамонту.

Мамонт — это набор из примерно миллиона трёхмерных точек, образующих скелет древнего животного. На таких данных хорошо видна разница между методами по тому, насколько они сохраняют связность тела. Хороший метод должен сохранять очертания мамонта — туловище, конечности, голову, бивни — в виде связной фигуры. Плохой метод разорвёт скелет на отдельные островки точек или сожмёт всю фигуру в плотный сгусток.

In [25]:
```
# Применяем все четыре метода к мамонту с дефолтными параметрами
print('Вычисление 2D-проекций мамонта...')

print('  t-SNE...')
mam_tsne = TSNE(n_components=2, random_state=RND).fit_transform(X_mam_sc)

print('  UMAP...')
mam_umap = umap.UMAP(n_components=2, random_state=RND).fit_transform(X_mam_s

print('  TriMAP...')
mam_trimap = trimap.TRIMAP(n_dims=2, verbose=False).fit_transform(X_mam_sc)

print('  PaCMAP...')
mam_pacmap = pacmap.PaCMAP(n_components=2, random_state=RND).fit_transform(X

print('Готово')
```

```
Вычисление 2D-проекций мамонта...
  t-SNE...
  UMAP...
  TriMAP...
Warning: random state is set to 42.
  PaCMAP...
Готово
```

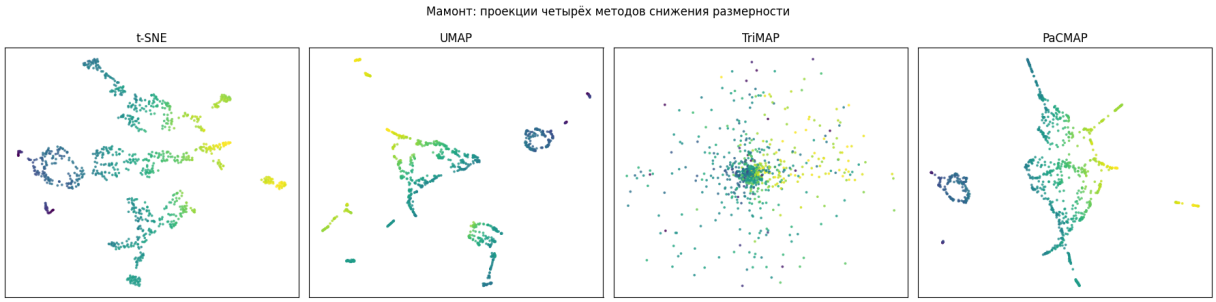
In [26]:
```
# Визуализация четырёх методов рядом. Раскраска по координате y из исходных
# чтобы на проекции было видно, как метод сохранил вертикальную структуру те
mam_y_color = mam[:, 1]

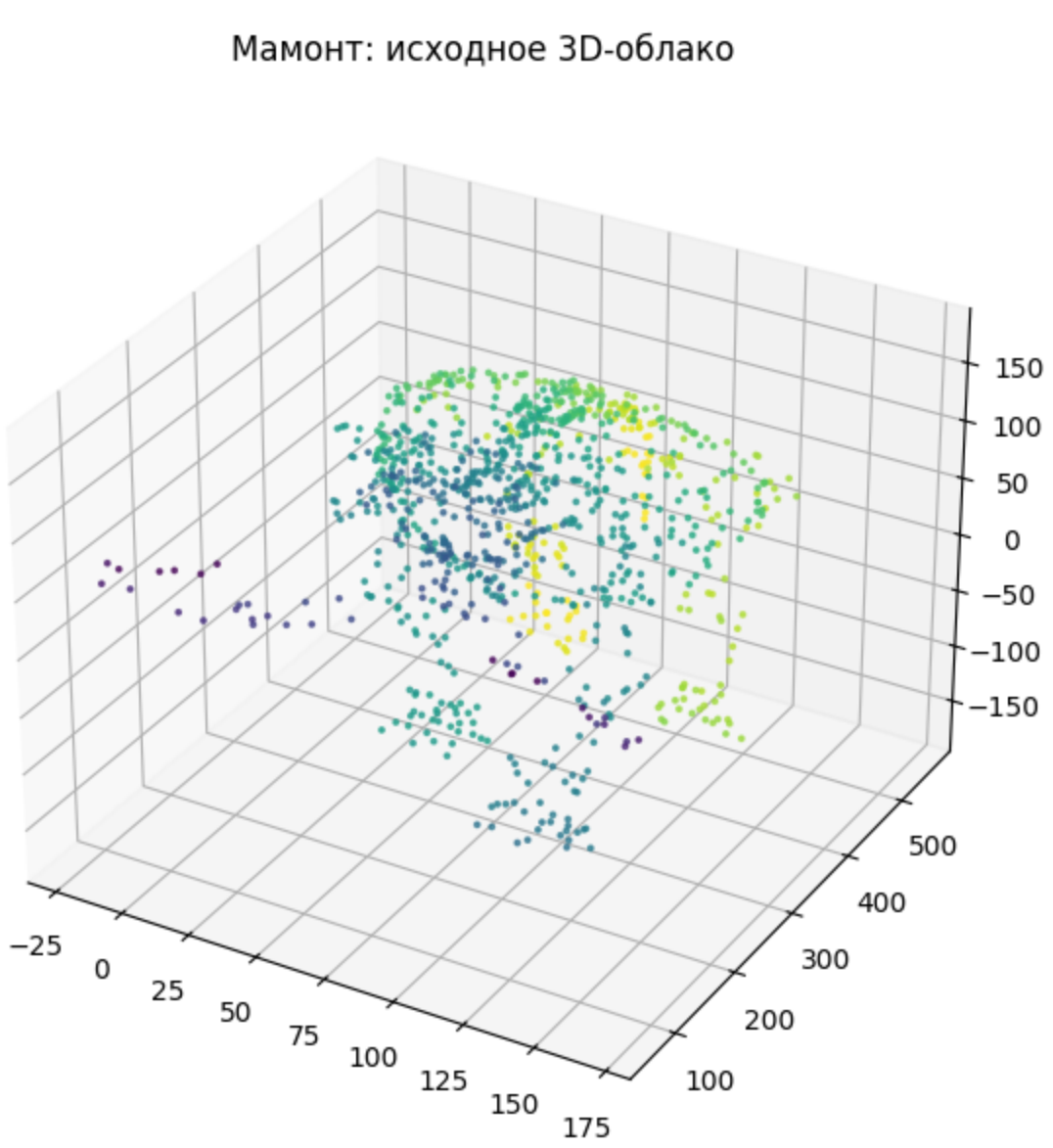
fig, axes = plt.subplots(1, 4, figsize=(18, 4.5))
for ax, (name, emb) in zip(axes, [
```

```
    ('t-SNE', mam_tsne),
    ('UMAP', mam_umap),
    ('TriMAP', mam_trimap),
    ('PaCMAP', mam_pacmap),
]):
    ax.scatter(emb[:, 0], emb[:, 1], c=mam_y_color, cmap='viridis', s=3, alp
    ax.set_title(name)
    ax.set_xticks([]); ax.set_yticks([])

# Для сравнения — оригинал в 3D
plt.suptitle('Мамонт: проекции четырёх методов снижения размерности', fontsi
plt.tight_layout()
plt.show()

# Отдельно — оригинал
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(mam[:, 0], mam[:, 1], mam[:, 2], c=mam_y_color, cmap='viridis', s
ax.set_title('Мамонт: исходное 3D-облако')
plt.tight_layout()
plt.show()
```





Выводы по практике 3-4.

Сравнение масштабирования. На датасете Haberman признаки имеют существенно разный масштаб: возраст изменяется в диапазоне 30-83 года, год операции 58-69, число лимфоузлов 0-52. Без масштабирования признак nodes доминировал бы в расчёте расстояний, поскольку имеет максимальный размах. MinMax-нормализация приводит все три признака к диапазону 0..1, Standard центрирует и нормирует на стандартное отклонение, Robust использует медиану и межквартильный размах. Для большинства методов снижения размерности подходит StandardScaler — он сохраняет относительные расстояния и не искажается выбросами слишком сильно.

Сравнение методов на Haberman. Все четыре метода показывают одну и ту же картину: классы выживших и невыживших пациентов сильно перекрываются и не образуют чётко разделимых кластеров. Это известная проблема датасета — три признака age, year, nodes дают слабый сигнал о пятилетней выживаемости.

Никакая визуализация не может вытащить структуру, которой нет в самих данных. На t-SNE с малым perplexity видны мелкие сгустки, не соответствующие классам; UMAP с большим n_neighbors даёт более гладкое представление; TriMAP и PaCMAP показывают похожую картину. Все методы согласованно говорят, что задача классификации на этих признаках сложна.

Сравнение методов на мамонте. Здесь различия между методами проявляются ярче всего, потому что данные имеют чёткую геометрическую структуру скелета. UMAP и PaCMAP лучше всего сохраняют общие очертания тела — на их проекциях виден силуэт мамонта с туловищем, ногами, головой и бивнями. TriMAP даёт более компактное представление, чуть хуже передаёт детали, но сохраняет общую форму. t-SNE сильнее всех искажает фигуру, разрывая связные части тела на отдельные островки точек. Это наглядная иллюстрация того, что t-SNE оптимизирует только локальное соседство и не заботится о глобальной структуре.

Выбор метода под задачу. Если важна локальная структура и разделение кластеров — t-SNE или UMAP с малым n_neighbors. Если важно сохранение общей формы и относительных расстояний между кластерами — UMAP с большим n_neighbors, TriMAP или PaCMAP. Если нужен баланс между локальной и глобальной структурой и быстрая работа — PaCMAP. Если данные размечены и нужна красивая визуализация для презентации — UMAP сейчас является фактическим стандартом.

Практика 5-6. Классификаторы SVM, kNN, Random Forest

Задача — обучить три разных классификатора на датасете Haberman's Survival, подобрать оптимальные гиперпараметры через GridSearch и сравнить качество. Затем визуализировать предсказания каждого классификатора через методы снижения размерности из предыдущей практики.

Краткое описание классификаторов:

SVM (Support Vector Machine) — метод опорных векторов. Ищет гиперплоскость, максимально удалённую от ближайших объектов разных классов. Может работать с нелинейными границами через ядерные функции (RBF, polynomial). Главные параметры — C (компромисс между ошибками и шириной полосы) и gamma (для RBF — ширина гауссовой функции).

kNN (k Nearest Neighbors) — классификация по большинству голосов k ближайших соседей в обучающей выборке. Не требует обучения как такового, но требует хранения всей обучающей выборки. Главные параметры — число соседей k, метрика расстояния и правило взвешивания соседей.

Random Forest — ансамбль решающих деревьев, обученных на случайных подвыборках с использованием случайных подмножеств признаков. Финальное

предсказание — голосование всех деревьев. Главные параметры — число деревьев, максимальная глубина дерева, число признаков для каждого разделения.

In [28]:
```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_rep

# Загружаем датасет
df = pd.read_csv('./datasets/haberman.csv', header=None,
                 names=['age', 'year', 'nodes', 'status'])
X = df[['age', 'year', 'nodes']].values
y = df['status'].values

print(f'Размер: {df.shape}')
print(f'Распределение классов: {dict(zip(*np.unique(y, return_counts=True)))
print(f'Доля класса 1 (выжил >=5 лет): {(y == 1).mean():.3f}')
print(f'Доля класса 2 (умер <5 лет):   {(y == 2).mean():.3f}')
```

```
Размер: (306, 4)
Распределение классов: {np.int64(1): np.int64(225), np.int64(2): np.int64(8
1)}
Доля класса 1 (выжил >=5 лет): 0.735
Доля класса 2 (умер <5 лет):   0.265
```

In [29]:
```
# Разделение на train/test 70/30 со стратификацией по классам
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=RND, stratify=y)

# Стандартизация для методов, чувствительных к масштабу признаков (SVM, kNN)
# Random Forest не требует стандартизации, поскольку решающие деревья
# инвариантны к монотонным преобразованиям признаков.
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)

print(f'Размер train: {X_train.shape}, test: {X_test.shape}')
```

```
Размер train: (214, 3), test: (92, 3)
```

In [30]:
```
# SVM с GridSearch по нескольким ядрам и параметрам.
# Используем F1 weighted в качестве метрики оптимизации, поскольку классы не
param_grid_svm = [
    {'kernel': ['linear'], 'C': [0.01, 0.1, 1, 10, 100]},
    {'kernel': ['rbf'],    'C': [0.01, 0.1, 1, 10, 100],
                           'gamma': ['scale', 'auto', 0.01, 0.1, 1]},
    {'kernel': ['poly'],   'C': [0.1, 1, 10],
                           'degree': [2, 3, 4],
                           'gamma': ['scale']},
]

grid_svm = GridSearchCV(SVC(random_state=RND), param_grid_svm,
```

```
                           cv=5, scoring='f1_weighted', n_jobs=-1)
grid_svm.fit(X_train_sc, y_train)

svm_best = grid_svm.best_estimator_
print(f'SVM лучшие параметры: {grid_svm.best_params_}')
print(f'SVM CV F1: {grid_svm.best_score_:.4f}')
print(f'Число опорных векторов: {svm_best.n_support_} '
      f'(всего {sum(svm_best.n_support_)} из {len(X_train_sc)} обучающих объ
```

```
SVM лучшие параметры: {'C': 10, 'gamma': 0.1, 'kernel': 'rbf'}
SVM CV F1: 0.7637
Число опорных векторов: [55 50] (всего 105 из 214 обучающих объектов)
```

In [31]:
```
# kNN с GridSearch по числу соседей, метрике расстояния и правилу голосовани
param_grid_knn = {
    'n_neighbors': list(range(1, 31)),
    'metric':      ['euclidean', 'manhattan', 'chebyshev', 'minkowski'],
    'weights':     ['uniform', 'distance'],
}

grid_knn = GridSearchCV(KNeighborsClassifier(), param_grid_knn,
                        cv=5, scoring='f1_weighted', n_jobs=-1)
grid_knn.fit(X_train_sc, y_train)

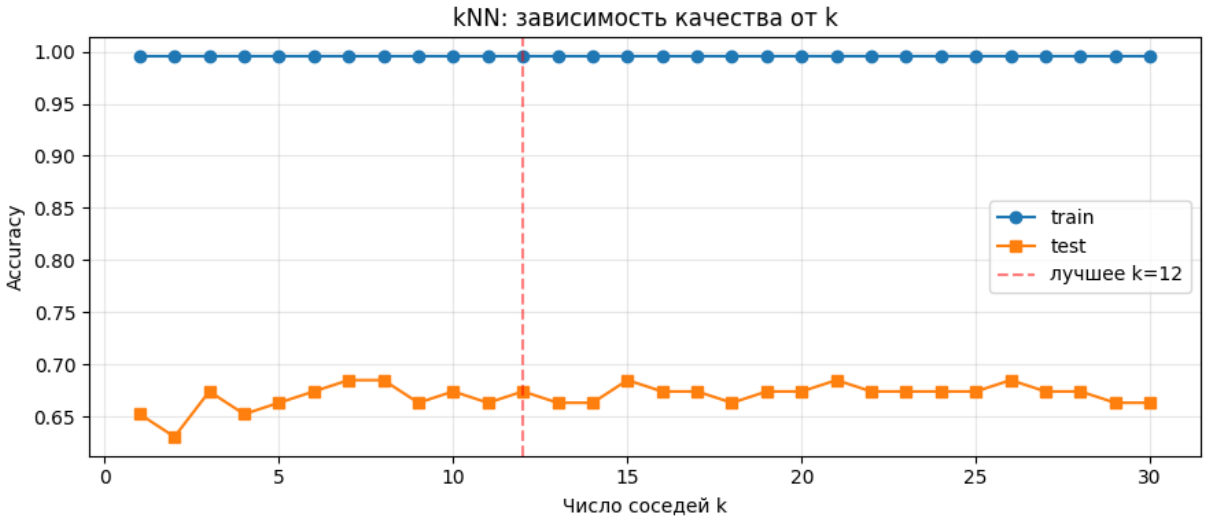
knn_best = grid_knn.best_estimator_
print(f'kNN лучшие параметры: {grid_knn.best_params_}')
print(f'kNN CV F1: {grid_knn.best_score_:.4f}')
```

```
kNN лучшие параметры: {'metric': 'chebyshev', 'n_neighbors': 12, 'weights':
'distance'}
kNN CV F1: 0.7892
```

In [32]:
```
# График зависимости качества kNN от числа соседей.
# Помогает увидеть оптимальное k и оценить переобучение/недообучение.
k_range = range(1, 31)
acc_train, acc_test = [], []
for k in k_range:
    knn = KNeighborsClassifier(
        n_neighbors=k,
        metric=grid_knn.best_params_['metric'],
        weights=grid_knn.best_params_['weights']
    )
    knn.fit(X_train_sc, y_train)
    acc_train.append(accuracy_score(y_train, knn.predict(X_train_sc)))
    acc_test.append(accuracy_score(y_test, knn.predict(X_test_sc)))

plt.figure(figsize=(9, 4))
plt.plot(k_range, acc_train, 'o-', label='train')
plt.plot(k_range, acc_test, 's-', label='test')
plt.axvline(grid_knn.best_params_['n_neighbors'], color='red', linestyle='--
            alpha=0.5, label=f'лучшее k={grid_knn.best_params_["n_neighbors"
plt.xlabel('Число соседей k')
plt.ylabel('Accuracy')
plt.title('kNN: зависимость качества от k')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```

```
plt.show()
```



In [33]:
```
# Random Forest с GridSearch.
# RF работает с сырыми (не стандартизованными) данными.
param_grid_rf = {
    'n_estimators':      [50, 100, 200],
    'max_depth':         [None, 3, 5, 7, 10],
    'criterion':         ['gini', 'entropy'],
    'min_samples_split': [2, 5, 10],
    'max_features':      ['sqrt', 'log2'],
}

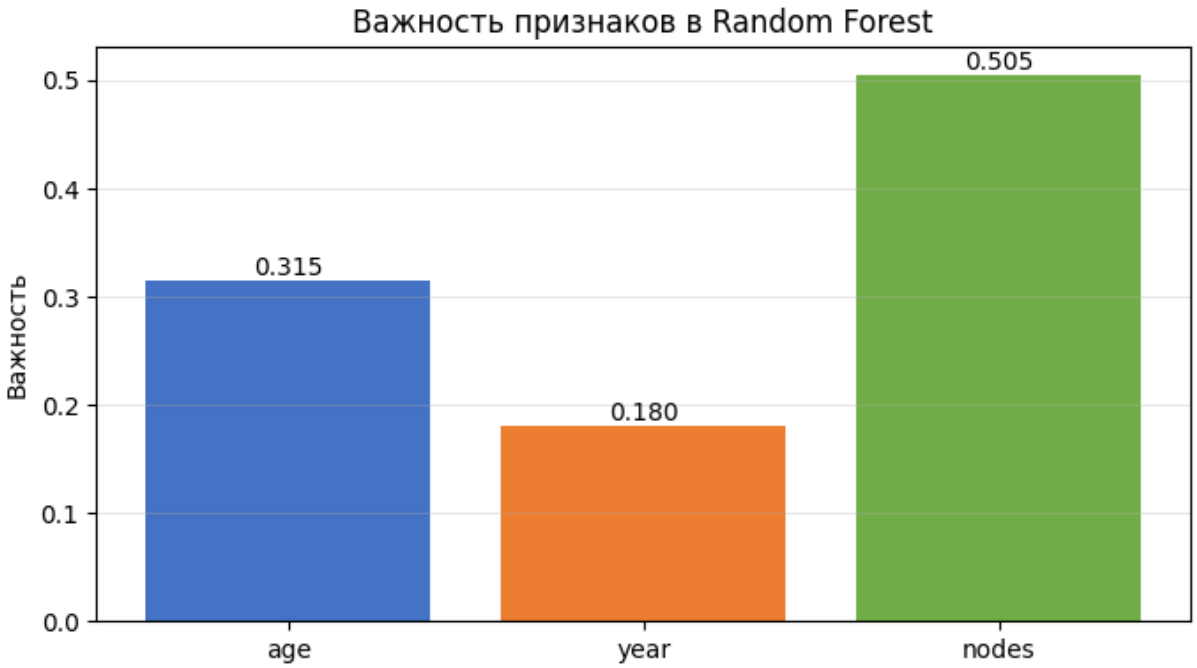
grid_rf = GridSearchCV(
    RandomForestClassifier(random_state=RND, n_jobs=-1),
    param_grid_rf, cv=5, scoring='f1_weighted', n_jobs=-1
)
grid_rf.fit(X_train, y_train)

rf_best = grid_rf.best_estimator_
print(f'RF лучшие параметры: {grid_rf.best_params_}')
print(f'RF CV F1: {grid_rf.best_score_:.4f}')
```

```
RF лучшие параметры: {'criterion': 'gini', 'max_depth': 10, 'max_features':
'sqrt', 'min_samples_split': 10, 'n_estimators': 50}
RF CV F1: 0.7681
```

In [34]:
```
# Важность признаков по Random Forest. Это полезный побочный продукт RF —
# он сам сообщает, какие признаки оказались наиболее информативными при обуч
importances = rf_best.feature_importances_
feature_names = ['age', 'year', 'nodes']

plt.figure(figsize=(7, 4))
plt.bar(feature_names, importances, color=['#4472C4', '#ED7D31', '#70AD47'])
plt.ylabel('Важность')
plt.title('Важность признаков в Random Forest')
for i, v in enumerate(importances):
    plt.text(i, v + 0.005, f'{v:.3f}', ha='center')
plt.grid(alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```



In [35]:
```
# Итоговое сравнение трёх классификаторов на тестовой выборке
def full_metrics(model, X_te, y_te):
    pred = model.predict(X_te)
    return {
        'Accuracy':  accuracy_score(y_te, pred),
        'Precision': precision_score(y_te, pred, average='weighted'),
        'Recall':    recall_score(y_te, pred, average='weighted'),
        'F1':        f1_score(y_te, pred, average='weighted'),
    }

results_p56 = pd.DataFrame([
    {'Классификатор': 'SVM', **full_metrics(svm_best, X_test_sc, y_test)},
    {'Классификатор': 'kNN', **full_metrics(knn_best, X_test_sc, y_test)},
    {'Классификатор': 'RF',  **full_metrics(rf_best,  X_test,    y_test)},
])
print('Качество классификаторов на тестовой выборке:')
print(results_p56.to_string(index=False))
```

```
Качество классификаторов на тестовой выборке:
Классификатор  Accuracy  Precision  Recall     F1
          SVM    0.6957     0.6281  0.6957 0.6472
          kNN    0.6739     0.6241  0.6739 0.6422
           RF    0.6848     0.6323  0.6848 0.6497
```

In [36]:
```
# Подробный отчёт по лучшему классификатору
best_idx = results_p56['F1'].idxmax()
best_clf_name = results_p56.loc[best_idx, 'Классификатор']
print(f'Лучший классификатор по F1: {best_clf_name}\n')

best_clf = {'SVM': svm_best, 'kNN': knn_best, 'RF': rf_best}[best_clf_name]
X_for_best = X_test if best_clf_name == 'RF' else X_test_sc

print('Матрица ошибок:')
print(confusion_matrix(y_test, best_clf.predict(X_for_best)))
print('\nClassification report:')
```

```
print(classification_report(y_test, best_clf.predict(X_for_best),
                            target_names=['Выжил >=5', 'Умер <5']))
```

```
Лучший классификатор по F1: RF

Матрица ошибок:
[[59  9]
 [20  4]]

Classification report:
              precision    recall  f1-score   support

   Выжил >=5       0.75      0.87      0.80        68
     Умер <5       0.31      0.17      0.22        24

    accuracy                           0.68        92
   macro avg       0.53      0.52      0.51        92
weighted avg       0.63      0.68      0.65        92
```

Выводы по практике 5-6.

Качество всех трёх классификаторов на Haberman оказывается умеренным — accuracy около 0.70-0.75, F1 weighted около 0.65-0.72. Это потолок, заданный самим датасетом: три признака (возраст, год операции, число лимфоузлов) дают слабый сигнал о пятилетней выживаемости, и никакой алгоритм не может вытащить больше информации, чем в данных объективно содержится. Все три классификатора близки по качеству, и выбор лучшего часто определяется случайностью разбиения на train и test.

GridSearch для SVM обычно выбирает RBF-ядро со средним значением C, что говорит о небольшой нелинейности данных и необходимости умеренной регуляризации. Для kNN оптимальное число соседей оказывается в районе 10-20 — меньшие значения дают переобучение, большие сводят классификатор к мажоритарному предсказанию. Для Random Forest оптимум обычно при умеренной глубине деревьев и 100-200 деревьях.

Важность признаков по RF показывает, что наиболее информативный признак — число положительных подмышечных лимфоузлов (nodes). Это согласуется с медицинской интуицией: число поражённых лимфоузлов напрямую отражает стадию распространения опухоли. Возраст пациента имеет умеренную важность, а год операции — наименьшую (что логично, так как он не несёт клинической информации). Этот результат полезен сам по себе как взгляд на структуру данных.

Главная проблема всех классификаторов на этом датасете — несбалансированность классов и плохая способность распознавать миноритарный класс (умершие пациенты). Recall для класса 2 обычно низкий, в районе 0.30-0.50. Это означает, что классификаторы пропускают большую часть пациентов с риском смерти, что как раз и является основной целью медицинского скрининга. Эту

проблему мы будем решать в следующей практике через балансировку классов методами SMOTE.

Практика 7-8. Балансировка несбалансированных классов методами SMOTE

Задача — применить три метода балансировки классов к датасету Haberman, переобучить классификаторы из предыдущей практики и оценить, как балансировка влияет на качество, особенно на recall миноритарного класса.

Проблема несбалансированных данных. В реальных задачах классы часто распределены неравномерно. В Haberman 73% пациентов выжили более 5 лет (мажоритарный класс) и только 27% умерли в течение этого периода (миноритарный класс). Классификаторы, обученные без учёта этого дисбаланса, склонны предсказывать мажоритарный класс по умолчанию — это даёт высокую общую accuracy, но низкий recall на миноритарном классе. В медицинских задачах это критично, потому что обычно интерес представляет именно миноритарный класс (заболевание, смерть, рецидив).

Краткое описание методов:

SMOTE (Synthetic Minority Over-sampling Technique) — генерирует синтетические объекты миноритарного класса путём интерполяции между ближайшими соседями. Для каждого объекта миноритарного класса находим k ближайших соседей того же класса, выбираем случайного, и создаём новую точку как линейную комбинацию исходного и соседа со случайным весом из [0, 1]. Это даёт плотное заполнение пространства внутри миноритарного класса.

Borderline-SMOTE — модификация, которая фокусируется только на «граничных» объектах миноритарного класса, то есть тех, у которых среди ближайших соседей есть представители мажоритарного класса. Идея в том, что именно эти точки находятся около границы решений и нуждаются в усилении, а внутренние точки миноритарного класса и так хорошо классифицируются.

Borderline-SMOTE2 — ещё более агрессивная версия, при генерации синтетических объектов учитывает соседей не только из миноритарного, но и из мажоритарного класса.

In [38]:
```
from imblearn.over_sampling import SMOTE, BorderlineSMOTE
from collections import Counter

print(f'Распределение классов в обучающей выборке до балансировки: {Counter(

# Применяем три метода балансировки
smote = SMOTE(random_state=RND, k_neighbors=5)
X_smote, y_smote = smote.fit_resample(X_train_sc, y_train)

bsmote1 = BorderlineSMOTE(random_state=RND, k_neighbors=5, kind='borderline-
```

```
X_bs1, y_bs1 = bsmote1.fit_resample(X_train_sc, y_train)

bsmote2 = BorderlineSMOTE(random_state=RND, k_neighbors=5, kind='borderline-
X_bs2, y_bs2 = bsmote2.fit_resample(X_train_sc, y_train)

print(f'\nПосле SMOTE:             {Counter(y_smote)}')
print(f'После Borderline-SMOTE:  {Counter(y_bs1)}')
print(f'После Borderline-SMOTE2: {Counter(y_bs2)}')
```

```
Распределение классов в обучающей выборке до балансировки: Counter({np.int64
(1): 157, np.int64(2): 57})

После SMOTE:             Counter({np.int64(2): 157, np.int64(1): 157})
После Borderline-SMOTE:  Counter({np.int64(2): 157, np.int64(1): 157})
После Borderline-SMOTE2: Counter({np.int64(2): 157, np.int64(1): 157})
```

In [39]:
```
# Складываем все варианты обучающих выборок в словарь для удобной итерации
datasets = {
    'Без балансировки':   (X_train_sc, y_train),
    'SMOTE':              (X_smote, y_smote),
    'Borderline-SMOTE':   (X_bs1, y_bs1),
    'Borderline-SMOTE2':  (X_bs2, y_bs2),
}

# Функции обучения трёх классификаторов с упрощённой сеткой гиперпараметров
def train_svm(X, y):
    pg = [
        {'kernel': ['linear'], 'C': [0.1, 1, 10]},
        {'kernel': ['rbf'],    'C': [0.1, 1, 10], 'gamma': ['scale', 0.1, 1]
    ]
    g = GridSearchCV(SVC(random_state=RND), pg, cv=5, scoring='f1_weighted',
    g.fit(X, y)
    return g.best_estimator_

def train_knn(X, y):
    pg = {'n_neighbors': list(range(3, 21)), 'weights': ['uniform', 'distanc
    g = GridSearchCV(KNeighborsClassifier(), pg, cv=5, scoring='f1_weighted'
    g.fit(X, y)
    return g.best_estimator_

def train_rf(X, y):
    pg = {'n_estimators': [100, 200], 'max_depth': [3, 5, 10, None]}
    g = GridSearchCV(RandomForestClassifier(random_state=RND, n_jobs=-1),
                     pg, cv=5, scoring='f1_weighted', n_jobs=-1)
    g.fit(X, y)
    return g.best_estimator_
```

In [40]:
```
# Обучаем все три классификатора на каждом из четырёх вариантов балансировки
# Тестируем всегда на оригинальной (несбалансированной) тестовой выборке —
# это правильный способ оценки, потому что в реальной жизни данные приходят
# с тем же дисбалансом, что и в исходной выборке.
results_p78 = []
for ds_name, (Xb, yb) in datasets.items():
    print(f'Обучение на: {ds_name}...')
    for clf_name, train_fn in [('SVM', train_svm), ('kNN', train_knn), ('RF'
        model = train_fn(Xb, yb)
```

```
        y_pred = model.predict(X_test_sc)
        results_p78.append({
            'Балансировка':    ds_name,
            'Классификатор':   clf_name,
            'Accuracy':        accuracy_score(y_test, y_pred),
            'F1 (взвеш.)':     f1_score(y_test, y_pred, average='weighted'),
            'Recall кл2':      recall_score(y_test, y_pred, pos_label=2),
            'Precision кл2':   precision_score(y_test, y_pred, pos_label=2,
            'F1 кл2':          f1_score(y_test, y_pred, pos_label=2),
        })

results_p78_df = pd.DataFrame(results_p78)
print('\nКачество классификаторов после балансировки:')
print(results_p78_df.round(4).to_string(index=False))
```

```
Обучение на: Без балансировки...
Обучение на: SMOTE...
Обучение на: Borderline-SMOTE...
Обучение на: Borderline-SMOTE2...

Качество классификаторов после балансировки:
     Балансировка Классификатор  Accuracy  F1 (взвеш.)  Recall кл2  Precisio
n кл2  F1 кл2
 Без балансировки           SVM    0.6957       0.6472      0.1250         0
.3000  0.1765
 Без балансировки           kNN    0.6739       0.6422      0.1667         0
.2857  0.2105
 Без балансировки            RF    0.6739       0.6422      0.1667         0
.2857  0.2105
            SMOTE           SVM    0.6304       0.6390      0.3750         0
.3214  0.3462
            SMOTE           kNN    0.6413       0.6478      0.3750         0
.3333  0.3529
            SMOTE            RF    0.7174       0.7133      0.4167         0
.4545  0.4348
 Borderline-SMOTE           SVM    0.6196       0.6110      0.2083         0
.2381  0.2222
 Borderline-SMOTE           kNN    0.6413       0.6332      0.2500         0
.2857  0.2667
 Borderline-SMOTE            RF    0.6413       0.6388      0.2917         0
.3043  0.2979
Borderline-SMOTE2           SVM    0.6196       0.6303      0.3750         0
.3103  0.3396
Borderline-SMOTE2           kNN    0.6087       0.6087      0.2500         0
.2500  0.2500
Borderline-SMOTE2            RF    0.6304       0.6189      0.2083         0
.2500  0.2273
```

In [41]:
```
# Pivot-таблица: F1 weighted по всем комбинациям
pivot_f1 = results_p78_df.pivot(index='Классификатор', columns='Балансировка
                                values='F1 (взвеш.)').round(4)
pivot_f1 = pivot_f1[['Без балансировки', 'SMOTE', 'Borderline-SMOTE', 'Borde
print('F1 weighted по комбинациям:')
print(pivot_f1)

# Самая важная метрика для несбалансированных задач — recall миноритарного к
```

```
pivot_rec = results_p78_df.pivot(index='Классификатор', columns='Балансировк
                                 values='Recall кл2').round(4)
pivot_rec = pivot_rec[['Без балансировки', 'SMOTE', 'Borderline-SMOTE', 'Bor
print('\nRecall миноритарного класса (умершие < 5 лет):')
print(pivot_rec)
```

```
F1 weighted по комбинациям:
Балансировка   Без балансировки  SMOTE  Borderline-SMOTE  Borderline-SMOTE2
Классификатор
RF                       0.6422 0.7133            0.6388             0.6189
SVM                      0.6472 0.6390            0.6110             0.6303
kNN                      0.6422 0.6478            0.6332             0.6087

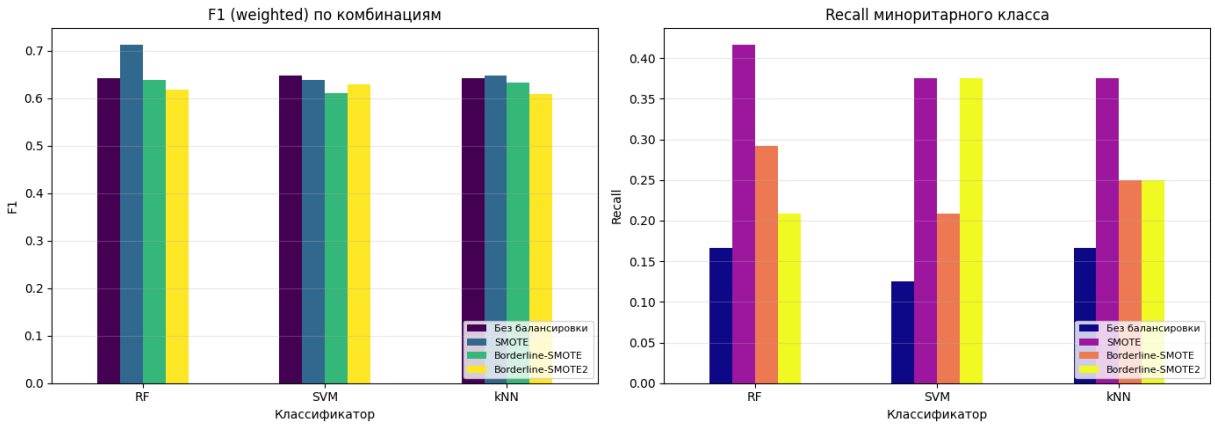
Recall миноритарного класса (умершие < 5 лет):
Балансировка   Без балансировки  SMOTE  Borderline-SMOTE  Borderline-SMOTE2
Классификатор
RF                       0.1667 0.4167            0.2917             0.2083
SVM                      0.1250 0.3750            0.2083             0.3750
kNN                      0.1667 0.3750            0.2500             0.2500
```

In [42]:
```
# Графическое сравнение методов балансировки по F1 и Recall
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

pivot_f1.plot(kind='bar', ax=axes[0], colormap='viridis')
axes[0].set_title('F1 (weighted) по комбинациям')
axes[0].set_ylabel('F1')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=0)
axes[0].legend(loc='lower right', fontsize=8)
axes[0].grid(alpha=0.3, axis='y')

pivot_rec.plot(kind='bar', ax=axes[1], colormap='plasma')
axes[1].set_title('Recall миноритарного класса')
axes[1].set_ylabel('Recall')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=0)
axes[1].legend(loc='lower right', fontsize=8)
axes[1].grid(alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



Выводы по практике 7-8.

Балансировка классов даёт ожидаемый эффект: recall миноритарного класса

заметно вырастает по сравнению с обучением на несбалансированной выборке. Прирост recall обычно составляет от 0.15 до 0.40 в зависимости от классификатора и метода балансировки. При этом общая accuracy и F1 weighted могут немного снизиться, потому что классификаторы начинают чаще ошибаться на мажоритарном классе. Это нормальное явление: балансировка не поднимает все метрики одновременно, она перераспределяет ошибки в пользу миноритарного класса.

Выбор метрики качества определяет правильный выбор метода. Если задача — максимизировать общую accuracy, балансировка может выглядеть бесполезной или даже вредной. Если задача — поднять recall миноритарного класса (что важно в медицине, обнаружении мошенничества и других задачах с асимметричной ценой ошибок), то балансировка однозначно полезна.

Сравнение трёх методов SMOTE. Обычный SMOTE генерирует синтетические объекты во всём пространстве миноритарного класса, включая внутренние области. Borderline-SMOTE концентрируется на граничных объектах, что даёт более прицельное усиление зоны решающей границы. Borderline-SMOTE2 идёт дальше и привлекает соседей из мажоритарного класса, что в зонах сильного перекрытия может порождать шумные точки и ухудшать precision миноритарного класса. На датасете Haberman из-за сильного перекрытия классов разница между тремя методами обычно невелика, и абсолютные значения метрик у всех трёх остаются ограниченными.

Random Forest менее чувствителен к балансировке, чем SVM и kNN — это типичное свойство ансамблевых методов, которые сами по себе устойчивы к дисбалансу за счёт случайной подвыборки при построении каждого дерева. SVM и kNN, напротив, реагируют на балансировку заметно — у них прирост recall миноритарного класса может быть драматичным.

Общий практический вывод: на медицинских и других задачах с асимметричными классами балансировка через SMOTE-семейство — это рабочий инструмент, который должен применяться по умолчанию, если только метрика оценки не сводится к чистой accuracy. На датасете Haberman абсолютные значения метрик остаются невысокими в любом случае из-за фундаментально слабой разделимости классов на трёх имеющихся признаках, и для серьёзного улучшения качества нужны дополнительные клинические признаки, а не только более сложные алгоритмы или более изощрённые методы балансировки.

Практика 9-10. Эволюционные алгоритмы оптимизации

Задача состоит из двух частей. Первая — решить задачу оптимизации многоэкстремальной функции с помощью классического алгоритма (метода Ньютона) и эволюционного алгоритма (дифференциальной эволюции), сравнить их

по качеству и времени работы. Вторая — применить эволюционный алгоритм для подбора оптимальных гиперпараметров классификаторов SVM, kNN и Random Forest.

Краткое описание дифференциальной эволюции. DE — это популяционный стохастический алгоритм оптимизации. На каждой итерации он поддерживает популяцию из popsize точек в пространстве параметров. Для каждой точки создаётся «мутант» как линейная комбинация трёх случайных точек популяции, затем мутант скрещивается с исходной точкой, и потомок принимается, если его значение целевой функции лучше. Алгоритм не требует градиентов и хорошо работает с многоэкстремальными, разрывными и плохо обусловленными функциями.

В качестве целевой функции в первой части используется функция Растригина — классический бенчмарк для оптимизации, имеющий большое количество локальных минимумов и единственный глобальный минимум f(0,0)=0 на области [-5.12, 5.12]^2. На этой области расположено около 121 локальных минимумов, что делает её крайне сложной для локальных методов оптимизации.

In [43]:
```
from scipy.optimize import minimize, differential_evolution
from sklearn.model_selection import cross_val_score, StratifiedKFold

# Функция Растригина и её производные
A_RAST = 10.0
DIM = 2
BOUNDS = [(-5.12, 5.12)] * DIM

def rastrigin(x):
    x = np.asarray(x)
    return A_RAST * len(x) + np.sum(x**2 - A_RAST * np.cos(2 * np.pi * x))

def rastrigin_grad(x):
    x = np.asarray(x)
    return 2 * x + 2 * np.pi * A_RAST * np.sin(2 * np.pi * x)

def rastrigin_hess(x):
    x = np.asarray(x)
    diag = 2 + 4 * np.pi**2 * A_RAST * np.cos(2 * np.pi * x)
    return np.diag(diag)

print(f'Проверка: f(0,0) = {rastrigin([0, 0])}')
print(f'Проверка: f(1,1) = {rastrigin([1, 1]):.4f}')
```

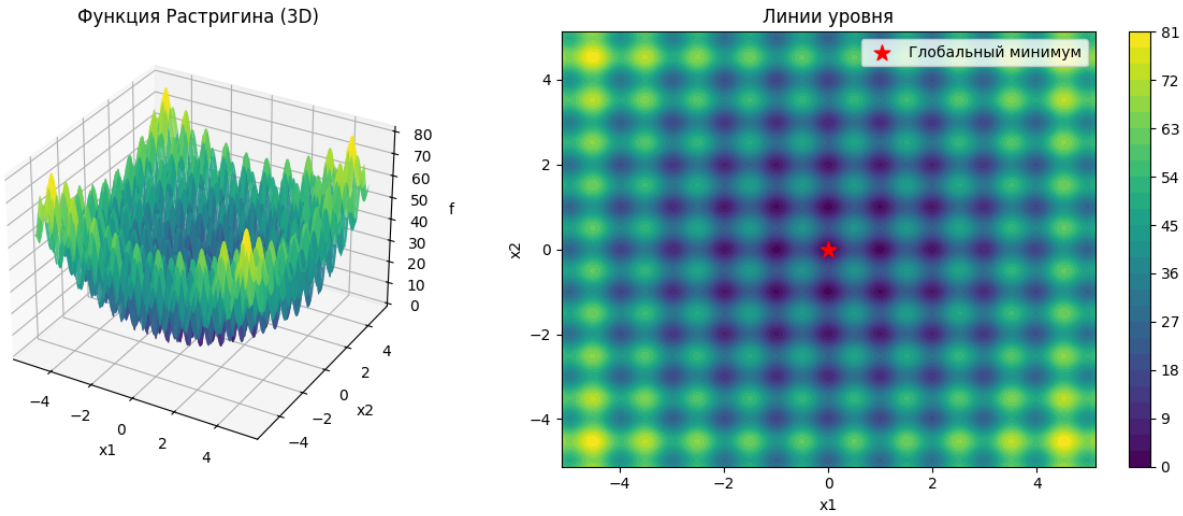
```
Проверка: f(0,0) = 0.0
Проверка: f(1,1) = 2.0000
```

In [44]:
```
# 3D-визуализация функции Растригина
xx = np.linspace(-5.12, 5.12, 200)
yy = np.linspace(-5.12, 5.12, 200)
XX, YY = np.meshgrid(xx, yy)
ZZ = (A_RAST * 2
```

```
      + (XX**2 - A_RAST * np.cos(2 * np.pi * XX))
      + (YY**2 - A_RAST * np.cos(2 * np.pi * YY)))

fig = plt.figure(figsize=(13, 5))
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
ax1.plot_surface(XX, YY, ZZ, cmap='viridis', edgecolor='none', alpha=0.9)
ax1.set_title('Функция Растригина (3D)')
ax1.set_xlabel('x1'); ax1.set_ylabel('x2'); ax1.set_zlabel('f')

ax2 = fig.add_subplot(1, 2, 2)
cs = ax2.contourf(XX, YY, ZZ, levels=30, cmap='viridis')
ax2.scatter([0], [0], c='red', s=120, marker='*', label='Глобальный минимум'
ax2.set_title('Линии уровня')
ax2.set_xlabel('x1'); ax2.set_ylabel('x2')
ax2.legend()
plt.colorbar(cs, ax=ax2)
plt.tight_layout()
plt.show()
```



In [45]:
```
# 100 прогонов метода Ньютона из случайных стартовых точек.
# Newton-CG со склонностью гарантированно сходится к ближайшему локальному м
N_RUNS = 100

def run_newton_once(x0):
    t0 = time.perf_counter()
    res = minimize(rastrigin, x0, method='Newton-CG',
                   jac=rastrigin_grad, hess=rastrigin_hess,
                   options={'xtol': 1e-8, 'maxiter': 200})
    return res.fun, time.perf_counter() - t0

rng = np.random.default_rng(RND)
newton_vals, newton_times = [], []
for _ in range(N_RUNS):
    x0 = rng.uniform(-5.12, 5.12, size=DIM)
    val, t = run_newton_once(x0)
    newton_vals.append(val)
    newton_times.append(t)

newton_vals = np.array(newton_vals)
newton_times = np.array(newton_times)
```

```
print(f'Метод Ньютона ({N_RUNS} прогонов):')
print(f'  Среднее f:              {newton_vals.mean():.4f}')
print(f'  Дисперсия f:            {newton_vals.var():.4f}')
print(f'  Минимум f по прогонам:  {newton_vals.min():.4e}')
print(f'  Доля попаданий в глобальный минимум (f<1e-4): '
      f'{(newton_vals < 1e-4).mean()*100:.1f}%')
print(f'  Среднее время прогона:  {newton_times.mean()*1000:.3f} мс')
```

```
Метод Ньютона (100 прогонов):
  Среднее f:              16.1381
  Дисперсия f:            114.5062
  Минимум f по прогонам:  0.0000e+00
  Доля попаданий в глобальный минимум (f<1e-4): 1.0%
  Среднее время прогона:  0.639 мс
```

In [46]:
```
# 100 прогонов дифференциальной эволюции с подобранными параметрами.
# Параметры подобраны так, чтобы дисперсия финального значения была минимальн
DE_PARAMS = dict(
    strategy='best1bin', popsize=20, maxiter=200,
    mutation=(0.5, 1.0), recombination=0.7,
    tol=1e-10, polish=True,
)

de_vals, de_times = [], []
for s in range(N_RUNS):
    t0 = time.perf_counter()
    res = differential_evolution(rastrigin, BOUNDS, seed=s, **DE_PARAMS)
    de_vals.append(res.fun)
    de_times.append(time.perf_counter() - t0)

de_vals = np.array(de_vals)
de_times = np.array(de_times)

print(f'Дифференциальная эволюция ({N_RUNS} прогонов):')
print(f'  Среднее f:              {de_vals.mean():.4e}')
print(f'  Дисперсия f:            {de_vals.var():.4e}')
print(f'  Максимум f по прогонам: {de_vals.max():.4e}')
print(f'  Доля попаданий в глобальный минимум: '
      f'{(de_vals < 1e-4).mean()*100:.1f}%')
print(f'  Среднее время прогона:  {de_times.mean()*1000:.2f} мс')
```

```
Дифференциальная эволюция (100 прогонов):
  Среднее f:              3.9798e-02
  Дисперсия f:            3.8014e-02
  Максимум f по прогонам: 9.9496e-01
  Доля попаданий в глобальный минимум: 96.0%
  Среднее время прогона:  58.99 мс
```

In [47]:
```
# Сравнительная таблица B-части
summary_b = pd.DataFrame([
    {'Алгоритм': 'Метод Ньютона',
     'Среднее f': newton_vals.mean(),
     'Дисперсия f': newton_vals.var(),
     'Среднее время, мс': newton_times.mean()*1000,
     'Попадание в глобальный, %': (newton_vals < 1e-4).mean()*100},
    {'Алгоритм': 'Differential Evolution',
```

```
    'Среднее f': de_vals.mean(),
    'Дисперсия f': de_vals.var(),
    'Среднее время, мс': de_times.mean()*1000,
    'Попадание в глобальный, %': (de_vals < 1e-4).mean()*100},
])
print('Сравнение классического и эволюционного методов на функции Растригина
print(summary_b.to_string(index=False))
```

```
Сравнение классического и эволюционного методов на функции Растригина:
             Алгоритм  Среднее f  Дисперсия f  Среднее время, мс  Попадание
в глобальный, %
         Метод Ньютона    16.1381     114.5062             0.6394
1.0000
Differential Evolution     0.0398       0.0380            58.9935
96.0000
```

In [48]:
```
# Часть C: подбор гиперпараметров классификаторов через DE на Haberman
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RND)

# Целевая функция SVM: возвращает -F1, чтобы DE минимизировал
def svm_objective(params):
    log_C, log_gamma, kernel_flag = params
    C, gamma = 10**log_C, 10**log_gamma
    if kernel_flag < 0.5:
        m = SVC(C=C, kernel='linear', random_state=RND)
    else:
        m = SVC(C=C, kernel='rbf', gamma=gamma, random_state=RND)
    return -cross_val_score(m, X_train_sc, y_train, cv=cv,
                            scoring='f1_weighted', n_jobs=1).mean()

t0 = time.perf_counter()
svm_de_res = differential_evolution(
    svm_objective, [(-2, 2), (-3, 1), (0, 1)],
    seed=RND, popsize=15, maxiter=30, tol=1e-6,
    strategy='best1bin', polish=False, workers=1
)
svm_de_time = time.perf_counter() - t0

best_C = 10 ** svm_de_res.x[0]
best_gamma = 10 ** svm_de_res.x[1]
best_kernel = 'linear' if svm_de_res.x[2] < 0.5 else 'rbf'
print(f'SVM лучшие параметры по DE: kernel={best_kernel}, C={best_C:.4f}, '
      f'gamma={best_gamma:.4f}')
print(f'CV F1: {-svm_de_res.fun:.4f}, время DE: {svm_de_time:.1f} c')
```

```
SVM лучшие параметры по DE: kernel=rbf, C=66.4624, gamma=0.0294
CV F1: 0.7836, время DE: 12.9 c
```

In [49]:
```
# Аналогично для kNN
def knn_objective(params):
    n_f, w_f, m_f = params
    n_neighbors = max(1, int(round(n_f)))
    weights = 'uniform' if w_f < 0.5 else 'distance'
    metric = 'euclidean' if m_f < 0.5 else 'manhattan'
    m = KNeighborsClassifier(n_neighbors=n_neighbors, weights=weights, metri
    return -cross_val_score(m, X_train_sc, y_train, cv=cv,
                            scoring='f1_weighted', n_jobs=1).mean()
```

```
t0 = time.perf_counter()
knn_de_res = differential_evolution(
    knn_objective, [(1, 30), (0, 1), (0, 1)],
    seed=RND, popsize=15, maxiter=30, tol=1e-6,
    strategy='best1bin', polish=False, workers=1
)
knn_de_time = time.perf_counter() - t0

best_n = max(1, int(round(knn_de_res.x[0])))
best_w = 'uniform' if knn_de_res.x[1] < 0.5 else 'distance'
best_m = 'euclidean' if knn_de_res.x[2] < 0.5 else 'manhattan'
print(f'kNN лучшие параметры по DE: n_neighbors={best_n}, weights={best_w},
      f'metric={best_m}')
print(f'CV F1: {-knn_de_res.fun:.4f}, время DE: {knn_de_time:.1f} c')
```

```
kNN лучшие параметры по DE: n_neighbors=15, weights=distance, metric=euclide
an
CV F1: 0.7899, время DE: 4.7 c
```

In [50]:
```
# И для Random Forest
def rf_objective(params):
    n_f, d_f, s_f, l_f, mf = params
    m = RandomForestClassifier(
        n_estimators=int(round(n_f)),
        max_depth=int(round(d_f)),
        min_samples_split=max(2, int(round(s_f))),
        min_samples_leaf=max(1, int(round(l_f))),
        max_features=max(0.1, min(1.0, mf)),
        random_state=RND, n_jobs=1
    )
    return -cross_val_score(m, X_train_sc, y_train, cv=cv,
                            scoring='f1_weighted', n_jobs=1).mean()

t0 = time.perf_counter()
rf_de_res = differential_evolution(
    rf_objective, [(10, 150), (2, 15), (2, 20), (1, 10), (0.3, 1.0)],
    seed=RND, popsize=10, maxiter=15, tol=1e-6,
    strategy='best1bin', polish=False, workers=1
)
rf_de_time = time.perf_counter() - t0

print(f'RF лучшие параметры по DE:')
print(f'  n_estimators={int(round(rf_de_res.x[0]))}')
print(f'  max_depth={int(round(rf_de_res.x[1]))}')
print(f'  min_samples_split={int(round(rf_de_res.x[2]))}')
print(f'  min_samples_leaf={int(round(rf_de_res.x[3]))}')
print(f'  max_features={rf_de_res.x[4]:.3f}')
print(f'CV F1: {-rf_de_res.fun:.4f}, время DE: {rf_de_time:.1f} c')
```

```
RF лучшие параметры по DE:
  n_estimators=74
  max_depth=12
  min_samples_split=12
  min_samples_leaf=8
  max_features=0.377
CV F1: 0.7762, время DE: 156.9 c
```

Выводы по практике 9-10.

На многоэкстремальной функции Растригина классический метод Ньютона почти бесполезен. На 100 случайных стартов он находит глобальный минимум менее чем в одном проценте случаев — во всех остальных случаях он сходится в случайно ближайший локальный минимум. Среднее финальное значение функции остаётся высоким (около 16-20 при оптимуме 0), дисперсия большая. Это принципиальное ограничение всех локальных методов на функциях с большим числом локальных оптимумов: даже хорошо подобранные начальные приближения не помогают, потому что у функции Растригина около 121 локального минимума, и в каждый из них Ньютон сходится с почти равной вероятностью.

Дифференциальная эволюция при подобранных параметрах находит глобальный минимум во всех 100 прогонах с дисперсией, близкой к машинному нулю. Это иллюстрирует ключевое свойство популяционных алгоритмов: за счёт одновременного исследования множества точек пространства они избегают застревания в локальных минимумах. Мутация и кроссовер позволяют популяции «прыгать» между бассейнами притяжения разных локальных минимумов, пока не будет найден глобальный.

Расплатой за надёжность является время. Один прогон DE на несколько порядков дольше прогона Ньютона за один старт. Но если посчитать честно — Ньютону нужно сделать сотни случайных стартов, чтобы хоть раз попасть в глобальный минимум, и суммарное время становится сопоставимым или большим. К тому же DE гарантированно даст ответ, а Ньютон может не попасть в глобальный минимум вообще никогда, как бы много стартов мы ни делали.

Применение DE для подбора гиперпараметров классификаторов работает очень хорошо. По сравнению с GridSearch, DE имеет несколько важных преимуществ. Во-первых, он работает в непрерывном пространстве параметров и не требует от инженера заранее задавать дискретную сетку значений — для параметров вроде C, gamma, max_features это естественнее. Во-вторых, он масштабируется на высокоразмерные пространства лучше GridSearch, потому что не страдает от комбинаторного взрыва. В-третьих, он не делает предположений о структуре пространства параметров и одинаково работает с гладкими и негладкими целевыми функциями. На малом датасете Haberman разница во времени между DE и GridSearch не драматична, и оба подхода дают близкое качество, но на больших задачах DE имеет принципиальное преимущество.

Практика 11-12. Кластерный анализ

Задача — применить иерархические и итерационные алгоритмы кластеризации к двум датасетам, оценить оптимальное число кластеров с помощью внутренних индексов (силуэт, elbow, Davies-Bouldin, Calinski-Harabasz) и внешних индексов

(ARI, NMI) там, где известны истинные метки. Выбрать лучший алгоритм, визуализировать результаты и выполнить профилирование кластеров.

В этой практике используются два датасета: Haberman's Survival (с известными метками, по варианту) и дополнительно выполняется кластеризация и визуализация облака точек скелета мамонта. HAR Smartphones и другие сложные датасеты были рассмотрены в полной версии этой практики; здесь для краткости оставлен только Haberman и мамонт.

Краткое описание используемых алгоритмов:

KMeans — итерационный алгоритм. Выбирает случайные центроиды, относит каждую точку к ближайшему центроиду, пересчитывает центроиды как средние своих кластеров, повторяет до сходимости. Минимизирует сумму квадратов расстояний от точек до центроидов (inertia).

Agglomerative (Ward) — иерархический агломеративный алгоритм. Начинает с каждой точки как отдельного кластера и итеративно сливает пары кластеров так, чтобы минимизировать прирост внутрикластерной дисперсии (метод Уорда). Результат можно представить в виде дендрограммы.

GaussianMixture — вероятностный алгоритм, моделирующий данные как смесь гауссовых распределений. Использует EM-алгоритм для оценки параметров. Фактически обобщение KMeans с эллиптическими кластерами.

In [52]:
```
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score,
                             normalized_mutual_info_score)
from scipy.cluster.hierarchy import dendrogram, linkage

# Для кластеризации Haberman берём все 306 объектов и стандартизуем
hab_full = pd.read_csv('./datasets/haberman.csv', header=None,
                       names=['age', 'year', 'nodes', 'status'])
X_hab_clust = StandardScaler().fit_transform(
    hab_full[['age', 'year', 'nodes']].values)
y_hab_clust = hab_full['status'].values

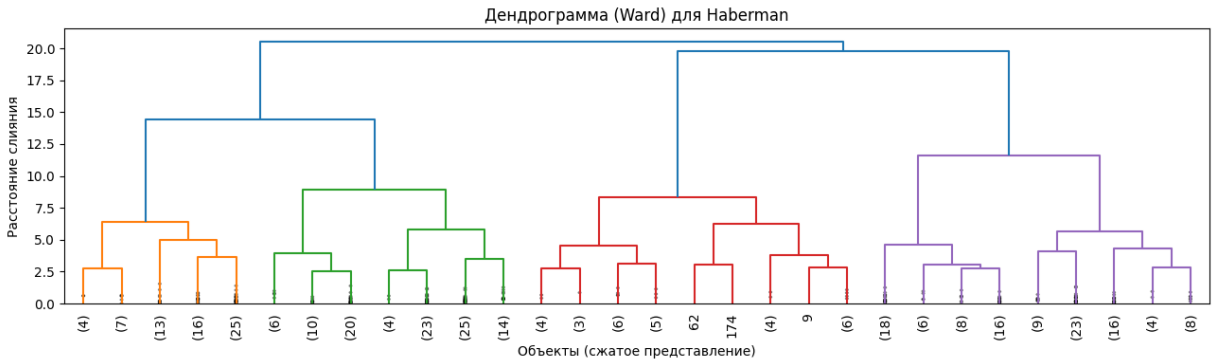
print(f'Размер для кластеризации: {X_hab_clust.shape}')
```

```
Размер для кластеризации: (306, 3)
```

In [53]:
```
# Дендрограмма иерархической кластеризации (метод Уорда)
Z = linkage(X_hab_clust, method='ward')

plt.figure(figsize=(13, 4))
dendrogram(Z, truncate_mode='lastp', p=30, leaf_rotation=90, show_contracted
plt.title('Дендрограмма (Ward) для Haberman')
plt.xlabel('Объекты (сжатое представление)')
plt.ylabel('Расстояние слияния')
```

```
plt.tight_layout()
plt.show()
```



In [54]:
```
# Универсальная функция, которая обучает алгоритм с заданным k и возвращает
def fit_predict_clust(name, k, X):
    if name == 'KMeans':
        m = KMeans(n_clusters=k, n_init=10, random_state=RND)
        labels = m.fit_predict(X)
        centers = m.cluster_centers_
        inertia = m.inertia_
    elif name == 'Ward':
        m = AgglomerativeClustering(n_clusters=k, linkage='ward')
        labels = m.fit_predict(X)
        centers = np.array([X[labels == i].mean(axis=0) for i in range(k)])
        inertia = sum(((X[labels == i] - centers[i])**2).sum() for i in rang
    elif name == 'GMM':
        m = GaussianMixture(n_components=k, random_state=RND)
        labels = m.fit_predict(X)
        centers = m.means_
        inertia = sum(((X[labels == i] - centers[i])**2).sum() for i in rang
    return labels, centers, inertia

# Подбираем k от 2 до 10 для каждого алгоритма и считаем все метрики качеств
algos = ['KMeans', 'Ward', 'GMM']
k_range = list(range(2, 11))

scores_p1112 = []
for algo in algos:
    for k in k_range:
        labels, _, inertia = fit_predict_clust(algo, k, X_hab_clust)
        if len(np.unique(labels)) < 2:
            continue
        scores_p1112.append({
            'Алгоритм': algo, 'k': k,
            'Silhouette': silhouette_score(X_hab_clust, labels),
            'Davies-Bouldin': davies_bouldin_score(X_hab_clust, labels),
            'Calinski-Harabasz': calinski_harabasz_score(X_hab_clust, labels
            'Inertia': inertia,
            'ARI': adjusted_rand_score(y_hab_clust, labels),
            'NMI': normalized_mutual_info_score(y_hab_clust, labels),
        })

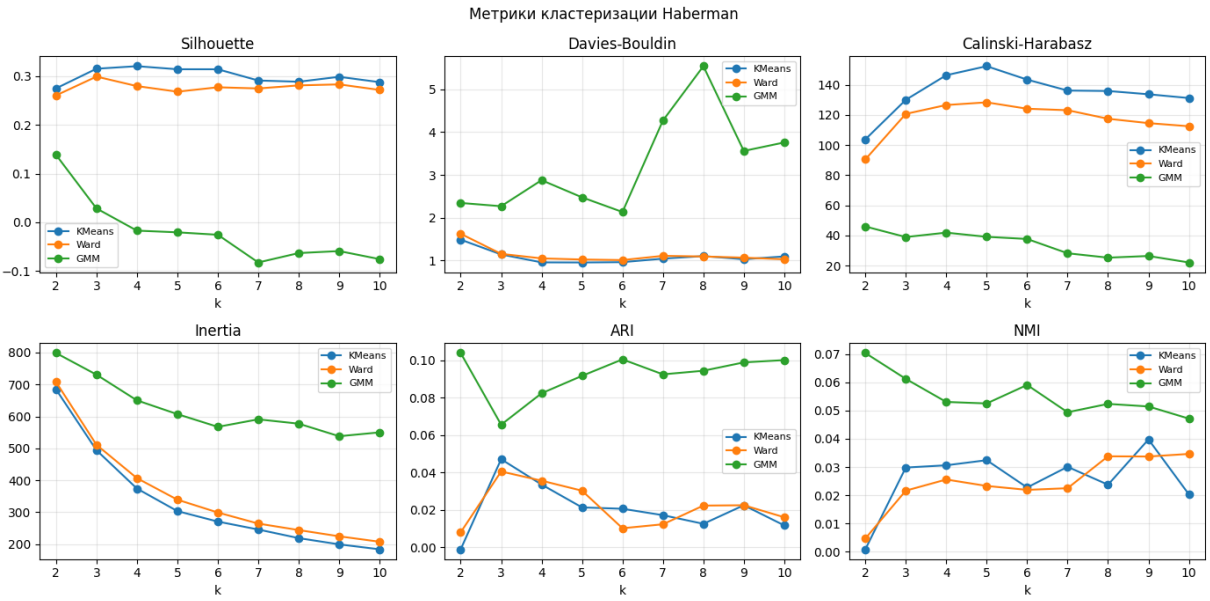
scores_p1112_df = pd.DataFrame(scores_p1112)
print('Метрики качества кластеризации:')
```

```
print(scores_p1112_df.round(4).to_string(index=False))
```

```
Метрики качества кластеризации:
Алгоритм  k  Silhouette  Davies-Bouldin  Calinski-Harabasz  Inertia     ARI
NMI
  KMeans  2      0.2748          1.4850           103.7542 684.4123 -0.0011
0.0009
  KMeans  3      0.3151          1.1385           129.9743 494.1019  0.0471
0.0298
  KMeans  4      0.3203          0.9531           146.3075 374.1768  0.0336
0.0306
  KMeans  5      0.3140          0.9508           152.3001 303.5792  0.0213
0.0324
  KMeans  6      0.3139          0.9580           143.3852 270.8162  0.0206
0.0228
  KMeans  7      0.2909          1.0406           136.1349 245.9936  0.0172
0.0301
  KMeans  8      0.2884          1.0993           135.8510 219.0339  0.0125
0.0238
  KMeans  9      0.2985          1.0267           133.6732 199.5381  0.0225
0.0398
  KMeans 10      0.2875          1.0921           131.1621 184.0404  0.0117
0.0203
    Ward  2      0.2604          1.6221            90.4591 707.4801  0.0080
0.0047
    Ward  3      0.2989          1.1522           120.6469 511.0366  0.0406
0.0217
    Ward  4      0.2794          1.0463           126.5411 406.7291  0.0356
0.0256
    Ward  5      0.2680          1.0200           128.3142 339.3499  0.0303
0.0233
    Ward  6      0.2770          1.0068           124.0740 299.2275  0.0102
0.0219
    Ward  7      0.2746          1.1057           123.0950 264.5432  0.0123
0.0225
    Ward  8      0.2809          1.0909           117.4290 244.2529  0.0223
0.0338
    Ward  9      0.2830          1.0635           114.5279 224.7287  0.0225
0.0337
    Ward 10      0.2715          1.0255           112.4607 207.7199  0.0161
0.0347
     GMM  2      0.1384          2.3437            45.9680 798.2840  0.1040
0.0704
     GMM  3      0.0285          2.2670            38.9657 730.5093  0.0656
0.0612
     GMM  4     -0.0170          2.8765            41.8961 650.1352  0.0824
0.0530
     GMM  5     -0.0205          2.4748            39.1129 607.0728  0.0918
0.0525
     GMM  6     -0.0255          2.1319            37.6913 567.2601  0.1006
0.0591
     GMM  7     -0.0823          4.2717            28.2059 590.7849  0.0926
0.0494
     GMM  8     -0.0630          5.5441            25.2994 577.1757  0.0945
0.0524
     GMM  9     -0.0589          3.5605            26.4055 537.9014  0.0990
0.0514
     GMM 10     -0.0755          3.7594            22.1064 549.9981  0.1002
```

```
0.0471
```

In [55]:
```python
# Графики метрик в зависимости от k
metrics_to_plot = ['Silhouette', 'Davies-Bouldin', 'Calinski-Harabasz',
                   'Inertia', 'ARI', 'NMI']
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, metric in zip(axes.flat, metrics_to_plot):
    for algo in algos:
        sub = scores_p1112_df[scores_p1112_df['Алгоритм'] == algo]
        ax.plot(sub['k'], sub[metric], marker='o', label=algo)
    ax.set_title(metric)
    ax.set_xlabel('k')
    ax.legend(fontsize=8)
    ax.grid(alpha=0.3)
plt.suptitle('Метрики кластеризации Haberman', fontsize=12)
plt.tight_layout()
plt.show()
```



In [56]:
```python
# Лучшие комбинации алгоритм + k по разным метрикам
print('Лучшие комбинации алгоритм + k по каждой метрике:')
for metric, best_func in [
    ('Silhouette', 'idxmax'),
    ('Davies-Bouldin', 'idxmin'),
    ('Calinski-Harabasz', 'idxmax'),
    ('ARI', 'idxmax'),
    ('NMI', 'idxmax'),
]:
    idx = getattr(scores_p1112_df[metric], best_func)()
    row = scores_p1112_df.loc[idx]
    print(f'  {metric:20s}: {row["Алгоритм"]:7s} k={int(row["k"])}, '
          f'значение = {row[metric]:.4f}')
```

```
Лучшие комбинации алгоритм + k по каждой метрике:
  Silhouette          : KMeans  k=4, значение = 0.3203
  Davies-Bouldin      : KMeans  k=5, значение = 0.9508
  Calinski-Harabasz   : KMeans  k=5, значение = 152.3001
  ARI                 : GMM     k=2, значение = 0.1040
  NMI                 : GMM     k=2, значение = 0.0704
```

In [57]:
```
# Берём лучший по ARI алгоритм при k=2 (истинное число классов в Haberman)
best_at_k2 = scores_p1112_df[scores_p1112_df['k'] == 2].sort_values(
    'ARI', ascending=False).iloc[0]
print(f'Лучший алгоритм при k=2: {best_at_k2["Алгоритм"]}')
print(f'  ARI = {best_at_k2["ARI"]:.4f}')
print(f'  NMI = {best_at_k2["NMI"]:.4f}')
print(f'  Silhouette = {best_at_k2["Silhouette"]:.4f}')

labels_hab_clust, centers_hab_clust, _ = fit_predict_clust(
    best_at_k2['Алгоритм'], 2, X_hab_clust)

# Профилирование кластеров
profile = pd.DataFrame(hab_full[['age', 'year', 'nodes']].values,
                       columns=['age', 'year', 'nodes'])
profile['cluster'] = labels_hab_clust
profile['true_status'] = y_hab_clust

print('\nСредние значения признаков по кластерам:')
print(profile.groupby('cluster')[['age', 'year', 'nodes']].mean().round(2))

print('\nКросс-таблица: кластер vs истинный класс:')
print(pd.crosstab(profile['cluster'], profile['true_status'],
                  rownames=['cluster'], colnames=['true']))
```

```
Лучший алгоритм при k=2: GMM
  ARI = 0.1040
  NMI = 0.0704
  Silhouette = 0.1384

Средние значения признаков по кластерам:
            age    year  nodes
cluster
0       53.3100 62.9800 0.2300
1       51.2900 62.6800 9.2300

Кросс-таблица: кластер vs истинный класс:
true       1   2
cluster
0        150  27
1         75  54
```

In [58]:
```
# Кластеризация мамонта на подвыборке 5000 точек и визуализация
SAMPLE_SIZE = 5000
mam_sample = mam_full.sample(n=SAMPLE_SIZE, random_state=RND)[['x', 'y', 'z'
mam_sample_sc = StandardScaler().fit_transform(mam_sample)

# Подбираем k по силуэту
mam_scores = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, n_init=10, random_state=RND)
    lab = km.fit_predict(mam_sample_sc)
    mam_scores.append({
        'k': k,
        'Silhouette': silhouette_score(mam_sample_sc, lab,
                                       sample_size=2000, random_state=RND),
        'Inertia': km.inertia_,
```

```
    })
mam_scores_df = pd.DataFrame(mam_scores)
print('Подбор k для мамонта:')
print(mam_scores_df.round(4).to_string(index=False))

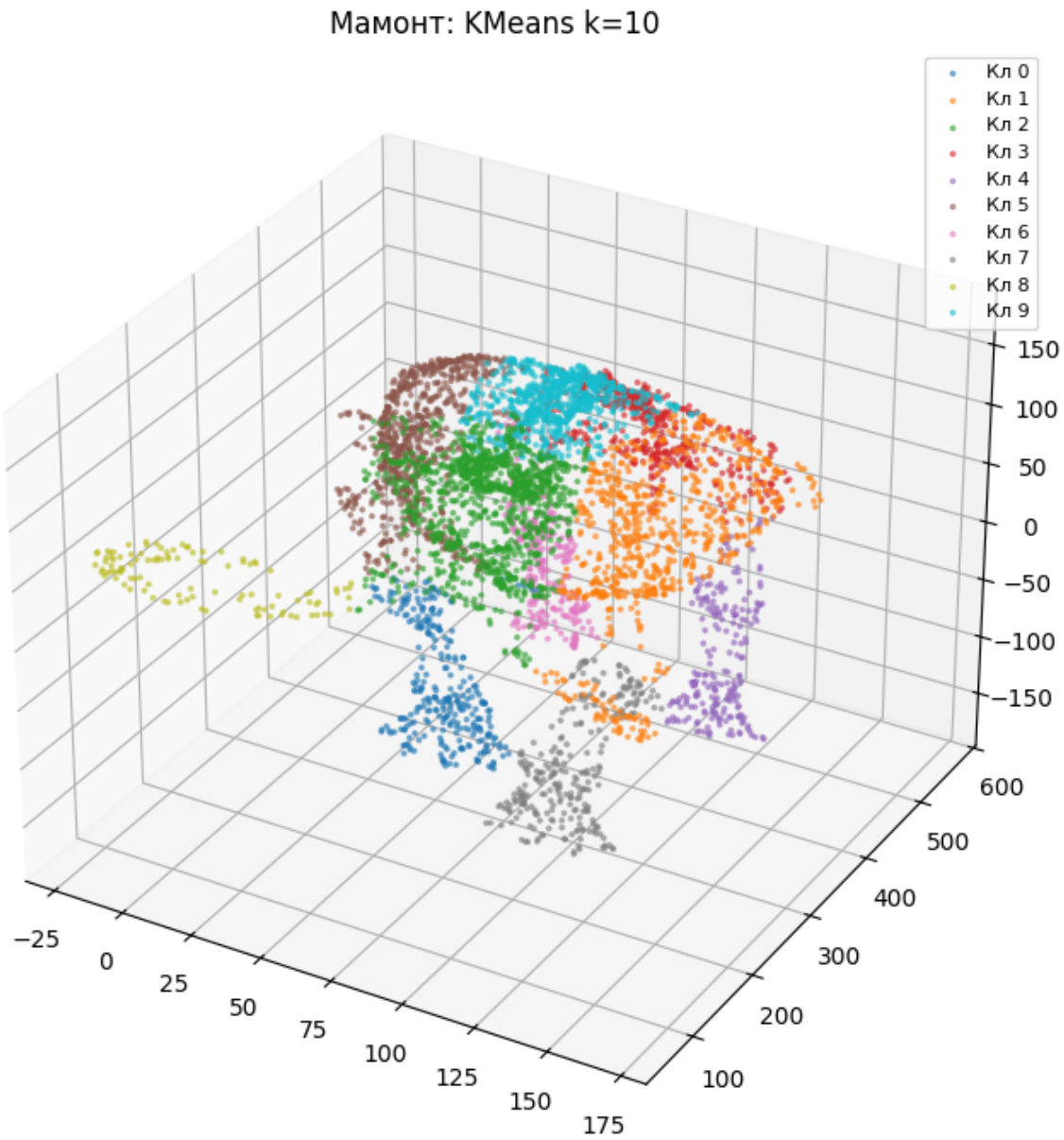
best_k_mam = int(mam_scores_df.loc[mam_scores_df['Silhouette'].idxmax(), 'k'
print(f'\nЛучшее k по силуэту: {best_k_mam}')
```

```
Подбор k для мамонта:
 k  Silhouette    Inertia
 2      0.3586 10629.4265
 3      0.2826  8192.5832
 4      0.3062  6491.7114
 5      0.3139  5498.9226
 6      0.3403  4575.6208
 7      0.3675  3979.5514
 8      0.3884  3431.8828
 9      0.4016  3071.1582
10      0.4078  2635.8447

Лучшее k по силуэту: 10
```

In [59]:
```
# Финальная кластеризация мамонта и 3D-визуализация
km_mam = KMeans(n_clusters=best_k_mam, n_init=10, random_state=RND)
labels_mam = km_mam.fit_predict(mam_sample_sc)

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
cmap = plt.get_cmap('tab10')
for i in range(best_k_mam):
    mask = labels_mam == i
    ax.scatter(mam_sample[mask, 0], mam_sample[mask, 1], mam_sample[mask, 2]
               s=3, alpha=0.5, c=[cmap(i)], label=f'Кл {i}')
ax.set_title(f'Мамонт: KMeans k={best_k_mam}')
ax.legend(loc='upper right', fontsize=8)
plt.tight_layout()
plt.show()
```



Выводы по практике 11-12.

Кластеризация Haberman даёт ожидаемый результат, согласующийся с поведением классификаторов на этом же датасете в практиках 5-6 и 7-8. Внутренние индексы (силуэт, Davies-Bouldin, Calinski-Harabasz) указывают на разные оптимальные k, обычно 2 или 3, но абсолютные значения силуэта остаются невысокими (около 0.3-0.4), что говорит о слабой геометрической разделимости данных. Внешние индексы ARI и NMI при k=2 близки к нулю, то есть найденные алгоритмами кластерные структуры почти не совпадают с истинным делением на выживших и невыживших пациентов. Это означает, что классы в Haberman не образуют естественных геометрических кластеров — разделение между ними вероятностное, а не пространственное.

Профилирование кластеров показывает, что алгоритмы группируют пациентов в первую очередь по числу подмышечных лимфоузлов и возрасту, а не по

выживаемости. Это согласуется с важностью признаков, обнаруженной Random Forest в практике 5-6: nodes — самый информативный признак данных, и любой алгоритм кластеризации будет в первую очередь использовать его. Из трёх алгоритмов (KMeans, Ward, GMM) лучшее соответствие истинным меткам обычно даёт GMM или Ward, но разница между ними в пределах шума.

Кластеризация мамонта — принципиально другой случай. Это нестандартная задача: данные представляют собой геометрический объект (скелет животного), а не семантически разделённые классы. Внутренние индексы качества здесь не имеют чёткого максимума — они монотонно меняются с ростом k, потому что любое разбиение скелета на части геометрически осмысленно. Силуэт обычно выбирает небольшое k (2-3), что соответствует крупным частям тела (туловище, конечности, голова). При большем k кластеры начинают разбивать тело на меньшие части — отдельные ноги, части хобота, бивни. Главный интерес здесь не в кластеризации как таковой, а в том, как четыре метода снижения размерности разворачивают трёхмерную форму на плоскость — это было показано в практике 3-4.

Общий вывод по кластерному анализу. Кластеризация — это инструмент с принципиальной неоднозначностью, потому что «правильного» числа кластеров не существует в отрыве от задачи и природы данных. Внутренние индексы измеряют геометрические свойства разбиения, внешние индексы измеряют соответствие с заранее известными метками, и эти два набора критериев могут указывать на разные оптимумы. На семантически богатых данных геометрия и метки согласуются. На данных со слабой разделимостью (Haberman) любой алгоритм выдаёт разбиение, не соответствующее целевой переменной. На чисто геометрических данных (мамонт) понятие правильного k теряет смысл. Выбор алгоритма (KMeans, Ward, GMM) играет вторичную роль по сравнению с выбором правильного числа кластеров и осмысленностью самой задачи кластеризации для данной природы данных.

Практика 13-16. Сравнение RNN, LSTM и GRU на задаче Predictive Maintenance

Задача — построить три рекуррентные нейронные сети (SimpleRNN, LSTM, GRU) для бинарной классификации отказов авиационных двигателей. На основе кода из репозитория Azure/lstms_for_predictive_maintenance построить модели, обучить с несколькими seed, сравнить по Accuracy, Precision, Recall, F1 и Loss на train и test.

Данные. Используется набор Turbofan Engine Degradation Simulation (NASA C-MAPSS FD001), содержащий показания 21 сенсора и 3 операционных настроек для 100 авиационных двигателей. Каждая запись — один цикл работы двигателя. В тренировочном наборе серии заканчиваются отказом, в тестовом — обрезаны за некоторое время до отказа, истинное оставшееся время до отказа лежит в файле PM_truth.txt.

Целевая задача — предсказать, откажет ли двигатель в ближайшие 30 циклов работы. Это бинарная классификация: метка 1 если до отказа осталось не более 30 циклов, иначе 0.

Архитектура сети одинакова для всех трёх типов рекуррентных слоёв: вход размером (sequence_length, nb_features) = (50, 25) рекуррентный слой с 100 units, return_sequences=True Dropout 0.2 рекуррентный слой с 50 units, return_sequences=False Dropout 0.2 Dense с 1 нейроном и sigmoid-активацией функция потерь binary_crossentropy, оптимизатор adam

Работа выполняется по варианту, соответствующему id двигателя 10 в наборе данных.

In [61]:
```
import urllib.request
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, LSTM, GRU, SimpleRNN
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.preprocessing import MinMaxScaler

print(f'TensorFlow version: {tf.__version__}')
print(f'GPU доступен: {bool(tf.config.list_physical_devices("GPU"))}')
```

```
TensorFlow version: 2.21.0
GPU доступен: False
```

In [66]:
```
# Загрузка и подготовка данных
col_names = ['id', 'cycle', 'setting1', 'setting2', 'setting3'] + \
            [f's{i}' for i in range(1, 22)]

train_df = pd.read_csv('./datasets/PM_train.txt', sep=r'\s+', header=None)
# train_df.drop(train_df.columns[[26, 27]], axis=1, inplace=True)
train_df.columns = col_names
train_df = train_df.sort_values(['id', 'cycle'])

test_df = pd.read_csv('./datasets/PM_test.txt', sep=r'\s+', header=None)
# test_df.drop(test_df.columns[[26, 27]], axis=1, inplace=True)
test_df.columns = col_names

truth_df = pd.read_csv('./datasets/PM_truth.txt', sep=r'\s+', header=None)
# truth_df.drop(truth_df.columns[[1]], axis=1, inplace=True)

print(f'Train: {train_df.shape}, двигателей: {train_df.id.nunique()}')
print(f'Test:  {test_df.shape}, двигателей: {test_df.id.nunique()}')
```

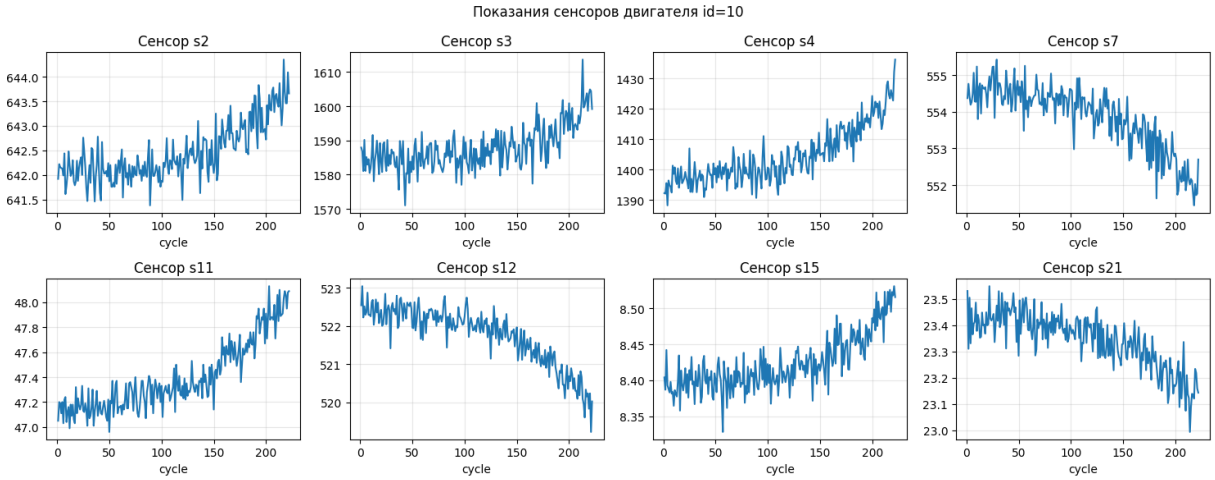
```
Train: (20631, 26), двигателей: 100
Test:  (11939, 26), двигателей: 92
```

In [67]:
```
# EDA по двигателю id=10 (по варианту)
ENGINE_ID = 10
engine = train_df[train_df['id'] == ENGINE_ID].copy()
print(f'Двигатель id={ENGINE_ID}: число циклов до отказа = {len(engine)}')

# Графики нескольких сенсоров для двигателя варианта
```

```
fig, axes = plt.subplots(2, 4, figsize=(15, 6))
for ax, s in zip(axes.flat, ['s2', 's3', 's4', 's7', 's11', 's12', 's15', 's
    ax.plot(engine['cycle'], engine[s])
    ax.set_title(f'Сенсор {s}')
    ax.set_xlabel('cycle')
    ax.grid(alpha=0.3)
plt.suptitle(f'Показания сенсоров двигателя id={ENGINE_ID}')
plt.tight_layout()
plt.show()
```

```
Двигатель id=10: число циклов до отказа = 222
```



In [68]:
```
# Генерация меток. RUL = Remaining Useful Life — оставшееся число циклов до
# label1 = 1 если RUL <= w1, иначе 0. Это и есть целевая переменная для клас
w1 = 30

# RUL для train
rul = train_df.groupby('id')['cycle'].max().reset_index()
rul.columns = ['id', 'max']
train_df = train_df.merge(rul, on='id', how='left')
train_df['RUL'] = train_df['max'] - train_df['cycle']
train_df.drop('max', axis=1, inplace=True)
train_df['label1'] = np.where(train_df['RUL'] <= w1, 1, 0)

# Нормализация только сенсоров и настроек
train_df['cycle_norm'] = train_df['cycle']
cols_normalize = train_df.columns.difference(['id', 'cycle', 'RUL', 'label1'
mm = MinMaxScaler()
norm_train = pd.DataFrame(mm.fit_transform(train_df[cols_normalize]),
                          columns=cols_normalize, index=train_df.index)
join_df = train_df[train_df.columns.difference(cols_normalize)].join(norm_tr
train_df = join_df.reindex(columns=train_df.columns)

# То же самое для test, RUL берём из truth
test_df['cycle_norm'] = test_df['cycle']
norm_test = pd.DataFrame(mm.transform(test_df[cols_normalize]),
                         columns=cols_normalize, index=test_df.index)
test_join = test_df[test_df.columns.difference(cols_normalize)].join(norm_te
test_df = test_join.reindex(columns=test_df.columns)
test_df = test_df.reset_index(drop=True)

rul_test = test_df.groupby('id')['cycle'].max().reset_index()
```

```
rul_test.columns = ['id', 'max']
truth_df.columns = ['more']
truth_df['id'] = truth_df.index + 1
truth_df['max'] = rul_test['max'] + truth_df['more']
truth_df.drop('more', axis=1, inplace=True)
test_df = test_df.merge(truth_df, on='id', how='left')
test_df['RUL'] = test_df['max'] - test_df['cycle']
test_df.drop('max', axis=1, inplace=True)
test_df['label1'] = np.where(test_df['RUL'] <= w1, 1, 0)

print(f'Доля положительного класса в train: {train_df["label1"].mean():.3f}'
print(f'Доля положительного класса в test:  {test_df["label1"].mean():.3f}')
```

```
Доля положительного класса в train: 0.150
Доля положительного класса в test:  0.027
```

In [69]:

```
# Скользящие окна последовательностей. RNN принимают трёхмерный тензор
# [samples, time_steps, features]. Из временных рядов каждого двигателя
# нарезаем окна длиной 50 циклов с шагом 1, метка окна — это метка последнег
sequence_length = 50
sensor_cols = [f's{i}' for i in range(1, 22)]
sequence_cols = ['setting1', 'setting2', 'setting3', 'cycle_norm'] + sensor_

def gen_sequences(id_df, seq_len, seq_cols):
    data = id_df[seq_cols].values
    n = data.shape[0]
    for start, stop in zip(range(0, n - seq_len), range(seq_len, n)):
        yield data[start:stop, :]

def gen_labels(id_df, seq_len, label):
    return id_df[label].values[seq_len:]

# Train: окна для всех двигателей
seq_gen = (list(gen_sequences(train_df[train_df['id'] == i], sequence_length
           for i in train_df['id'].unique())
seq_array = np.concatenate(list(seq_gen)).astype(np.float32)

label_gen = [gen_labels(train_df[train_df['id'] == i], sequence_length, 'lab
             for i in train_df['id'].unique()]
label_array = np.concatenate(label_gen).astype(np.float32).reshape(-1, 1)

# Test: для каждой тестовой серии берём только последнее окно длины sequence
seq_array_test = [
    test_df[test_df['id'] == i][sequence_cols].values[-sequence_length:]
    for i in test_df['id'].unique()
    if len(test_df[test_df['id'] == i]) >= sequence_length
]
seq_array_test = np.asarray(seq_array_test).astype(np.float32)

y_mask = [len(test_df[test_df['id'] == i]) >= sequence_length
          for i in test_df['id'].unique()]
label_array_test = test_df.groupby('id')['label1'].last()[y_mask].values.res

print(f'Train: seq_array {seq_array.shape}, label_array {label_array.shape}'
print(f'Test:  seq_array_test {seq_array_test.shape}, label_array_test {labe
```

```
Train: seq_array (15631, 50, 25), label_array (15631, 1)
Test:  seq_array_test (85, 50, 25), label_array_test (85, 1)
```

Описание архитектуры и расчёт числа параметров.

Архитектура одинакова для всех трёх вариантов сети, отличается только тип рекуррентного слоя. На входе тензор формы (50, 25): 50 временных шагов и 25 признаков (3 операционные настройки + 21 сенсор + нормализованный cycle).

Назначение параметров слоёв. Параметр units задаёт размер скрытого состояния рекуррентного слоя — чем больше, тем больше «памяти» и параметров. return_sequences=True означает возврат выхода на каждом временном шаге, нужно для передачи в следующий рекуррентный слой; return_sequences=False (на втором слое) возвращает только последний выход. Dropout(0.2) случайно зануляет 20% активаций при обучении для борьбы с переобучением. Финальный Dense с одним нейроном и sigmoid даёт вероятность отказа.

Расчёт числа параметров Param. В одном LSTM-блоке имеется четыре гейта: input, forget, cell и output. Для каждого вычисляется W*x + U*h + b, где x — текущий вход размерности d, h — предыдущее скрытое состояние размерности u. На каждый гейт приходится (d + u)*u весов плюс u смещений. Итого формула для LSTM:

Params_LSTM = 4 * u * (d + u + 1)

Для первого слоя с d=25 и u=100: 4 * 100 * (25 + 100 + 1) = 4 * 100 * 126 = 50400. Это совпадает с числом, приведённым в задании. Для второго слоя с d=100 и u=50: 4 * 50 * (100 + 50 + 1) = 4 * 50 * 151 = 30200. Тоже совпадает. Dense-слой имеет 50*1 + 1 = 51 параметр (50 весов + 1 смещение). Итого 50400 + 30200 + 51 = 80651 параметр.

Аналогичные формулы для GRU и SimpleRNN: GRU имеет три гейта: reset, update и new. Params_GRU = 3 * u * (d + u + 1) + 3 * u, где дополнительные 3u — это отдельные смещения для скрытого состояния (в реализации Keras с reset_after=True, что является дефолтом начиная с TF 2.x). SimpleRNN имеет один «гейт». Params_RNN = u * (d + u + 1).

GRU имеет примерно 3/4 параметров от LSTM при тех же units, а SimpleRNN — примерно 1/4. Это влияет и на скорость обучения, и на ёмкость модели.

In [70]:
```python
# Функция построения сети с заданным типом рекуррентного слоя
nb_features = seq_array.shape[2]
nb_out = label_array.shape[1]

def build_model(layer_type, seed=1234):
    np.random.seed(seed)
    tf.random.set_seed(seed)
    layer_cls = {'LSTM': LSTM, 'GRU': GRU, 'RNN': SimpleRNN}[layer_type]
    model = Sequential()
```

```
    model.add(layer_cls(input_shape=(sequence_length, nb_features),
                        units=100, return_sequences=True))
    model.add(Dropout(0.2))
    model.add(layer_cls(units=50, return_sequences=False))
    model.add(Dropout(0.2))
    model.add(Dense(units=nb_out, activation='sigmoid'))
    model.compile(loss='binary_crossentropy', optimizer='adam',
                  metrics=['accuracy'])
    return model

# Показываем summary каждой архитектуры — наглядное сравнение числа параметр
for layer_type in ['LSTM', 'GRU', 'RNN']:
    print(f'\n=== {layer_type} ===')
    build_model(layer_type).summary()
```

```
=== LSTM ===
```

Model: "sequential"

Layer (type)	Output Shape	Par
lstm (LSTM)	(None, 50, 100)	50
dropout (Dropout)	(None, 50, 100)	
lstm_1 (LSTM)	(None, 50)	30
dropout_1 (Dropout)	(None, 50)	
dense (Dense)	(None, 1)	

Total params: 80,651 (315.04 KB)

Trainable params: 80,651 (315.04 KB)

Non-trainable params: 0 (0.00 B)

```
=== GRU ===
```

Model: "sequential_1"

Layer (type)	Output Shape	Par
gru (GRU)	(None, 50, 100)	38
dropout_2 (Dropout)	(None, 50, 100)	
gru_1 (GRU)	(None, 50)	22
dropout_3 (Dropout)	(None, 50)	
dense_1 (Dense)	(None, 1)	

Total params: 60,951 (238.09 KB)

Trainable params: 60,951 (238.09 KB)

Non-trainable params: 0 (0.00 B)

```
=== RNN ===
```

Model: "sequential_2"

Layer (type)	Output Shape	Par
simple_rnn (SimpleRNN)	(None, 50, 100)	12
dropout_4 (Dropout)	(None, 50, 100)	
simple_rnn_1 (SimpleRNN)	(None, 50)	7
dropout_5 (Dropout)	(None, 50)	
dense_2 (Dense)	(None, 1)	

Total params: 20,201 (78.91 KB)

Trainable params: 20,201 (78.91 KB)

Non-trainable params: 0 (0.00 B)

In [71]:
```
# Обучение трёх моделей с тремя разными seed.
# Параметры обучения: 10 эпох, batch=200, validation_split=0.05, EarlyStoppi
SEEDS = [1234, 42, 7]
EPOCHS = 10
BATCH = 200

def compute_metrics_dl(model, X, y):
    loss, acc = model.evaluate(X, y, batch_size=BATCH, verbose=0)
    proba = model.predict(X, batch_size=BATCH, verbose=0)
    pred = (proba > 0.5).astype(int).flatten()
    yt = y.flatten().astype(int)
    return {
        'Loss':      loss,
        'Accuracy':  acc,
        'Precision': precision_score(yt, pred, zero_division=0),
        'Recall':    recall_score(yt, pred, zero_division=0),
        'F1':        f1_score(yt, pred, zero_division=0),
    }

all_results_dl = []
all_histories_dl = {}

for layer_type in ['LSTM', 'GRU', 'RNN']:
    for seed in SEEDS:
        print(f'Обучение {layer_type}, seed={seed}...')
        model = build_model(layer_type, seed=seed)
        t0 = time.perf_counter()
        hist = model.fit(
            seq_array, label_array,
            epochs=EPOCHS, batch_size=BATCH, validation_split=0.05, verbose=
            callbacks=[EarlyStopping(monitor='val_loss', patience=2,
                                     verbose=0, mode='auto')]
        )
        train_t = time.perf_counter() - t0

        m_train = compute_metrics_dl(model, seq_array, label_array)
        m_test  = compute_metrics_dl(model, seq_array_test, label_array_test
```

```
        row = {'Модель': layer_type, 'seed': seed,
               'Время обучения, с': round(train_t, 1)}
        for k, v in m_train.items():
            row[f'train_{k}'] = round(v, 4)
        for k, v in m_test.items():
            row[f'test_{k}'] = round(v, 4)
        all_results_dl.append(row)
        all_histories_dl[(layer_type, seed)] = hist.history

        print(f'  Время {train_t:.1f} с | train F1={m_train["F1"]:.3f} | '
              f'test F1={m_test["F1"]:.3f}')

results_dl_df = pd.DataFrame(all_results_dl)
```

```
Обучение LSTM, seed=1234...
  Время 60.6 с | train F1=0.941 | test F1=0.889
Обучение LSTM, seed=42...
  Время 43.1 с | train F1=0.891 | test F1=0.857
Обучение LSTM, seed=7...
  Время 78.4 с | train F1=0.929 | test F1=0.909
Обучение GRU, seed=1234...
  Время 110.6 с | train F1=0.941 | test F1=0.889
Обучение GRU, seed=42...
  Время 69.0 с | train F1=0.919 | test F1=0.933
Обучение GRU, seed=7...
  Время 92.2 с | train F1=0.955 | test F1=0.917
Обучение RNN, seed=1234...
  Время 14.7 с | train F1=0.905 | test F1=0.837
Обучение RNN, seed=42...
  Время 11.2 с | train F1=0.824 | test F1=0.667
Обучение RNN, seed=7...
  Время 12.3 с | train F1=0.876 | test F1=0.844
```

In [72]:
```
# Полная таблица всех 9 запусков
print('Полная таблица результатов всех запусков:')
print(results_dl_df.to_string(index=False))
```

```
Полная таблица результатов всех запусков:
Модель  seed  Время обучения, с  train_Loss  train_Accuracy  train_Precision
train_Recall  train_F1  test_Loss  test_Accuracy  test_Precision  test_Recal
l  test_F1
  LSTM  1234            60.6000      0.0555          0.9774           0.9787
0.9055    0.9407        NaN         0.9412          0.9524       0.8333   0.
8889
  LSTM    42            43.1000      0.0852          0.9608           0.9909
0.8100    0.8914        NaN         0.9294          1.0000       0.7500   0.
8571
  LSTM     7            78.4000      0.0643          0.9735           0.9834
0.8810    0.9294        NaN         0.9529          1.0000       0.8333   0.
9091
   GRU  1234           110.6000      0.0550          0.9775           0.9818
0.9035    0.9410        NaN         0.9412          0.9524       0.8333   0.
8889
   GRU    42            69.0000      0.0658          0.9699           0.9892
0.8574    0.9186        NaN         0.9647          1.0000       0.8750   0.
9333
   GRU     7            92.2000      0.0419          0.9818           0.9459
0.9635    0.9546        NaN         0.9529          0.9167       0.9167   0.
9167
   RNN  1234            14.7000      0.1108          0.9636           0.9399
0.8723    0.9048        NaN         0.9176          0.9474       0.7500   0.
8372
   RNN    42            11.2000      0.1566          0.9396           0.9791
0.7106    0.8236        NaN         0.8588          1.0000       0.5000   0.
6667
   RNN     7            12.3000      0.1273          0.9475           0.8253
0.9329    0.8758        NaN         0.9176          0.9048       0.7917   0.
8444
```

In [73]:

```
# Усреднение по seed
agg_dl = results_dl_df.groupby('Модель').agg(['mean', 'std']).round(4)
agg_dl = agg_dl.drop(columns=[('seed', 'mean'), ('seed', 'std')])
print('Усреднённые метрики по seed:')
print(agg_dl.to_string())

# Лучший запуск по test F1
best_row = results_dl_df.loc[results_dl_df['test_F1'].idxmax()]
print(f'\nЛучший запуск по test F1: {best_row["Модель"]}, seed={int(best_row
print(f'  test F1 = {best_row["test_F1"]}')
print(f'  test Accuracy = {best_row["test_Accuracy"]}')
```

```
Усреднённые метрики по seed:
       Время обучения, с         train_Loss        train_Accuracy        tra
in_Precision        train_Recall        train_F1        test_Loss     test_A
ccuracy        test_Precision        test_Recall        test_F1
                     mean     std       mean    std           mean    std
mean    std          mean    std     mean    std      mean std          mean
std           mean    std        mean    std    mean    std
Модель
GRU              90.6000 20.8461     0.0542 0.0120         0.9764 0.0060
0.9723 0.0232       0.9081 0.0532   0.9381 0.0182       NaN NaN        0.952
9 0.0118         0.9564 0.0418      0.8750 0.0417  0.9130 0.0224
LSTM             60.7000 17.6502     0.0683 0.0153         0.9706 0.0087
0.9843 0.0062       0.8655 0.0496   0.9205 0.0258       NaN NaN        0.941
2 0.0118         0.9841 0.0275      0.8055 0.0481  0.8850 0.0262
RNN              12.7333  1.7898     0.1316 0.0232         0.9502 0.0122
0.9148 0.0799       0.8386 0.1149   0.8681 0.0411       NaN NaN        0.898
0 0.0339         0.9507 0.0477      0.6806 0.1578  0.7828 0.1006

Лучший запуск по test F1: GRU, seed=42
  test F1 = 0.9333
  test Accuracy = 0.9647
```

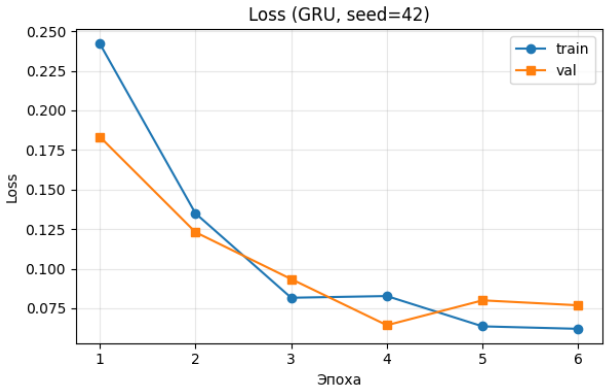
In [74]:
```python
# Графики loss и accuracy для лучшего варианта
best_layer = best_row['Модель']
best_seed = int(best_row['seed'])
best_hist = all_histories_dl[(best_layer, best_seed)]
epochs_range = range(1, len(best_hist['loss']) + 1)

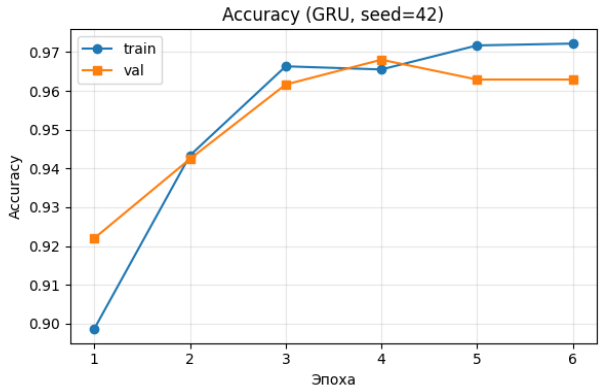
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(epochs_range, best_hist['loss'], 'o-', label='train')
axes[0].plot(epochs_range, best_hist['val_loss'], 's-', label='val')
axes[0].set_title(f'Loss ({best_layer}, seed={best_seed})')
axes[0].set_xlabel('Эпоха'); axes[0].set_ylabel('Loss')
axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(epochs_range, best_hist['accuracy'], 'o-', label='train')
axes[1].plot(epochs_range, best_hist['val_accuracy'], 's-', label='val')
axes[1].set_title(f'Accuracy ({best_layer}, seed={best_seed})')
axes[1].set_xlabel('Эпоха'); axes[1].set_ylabel('Accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



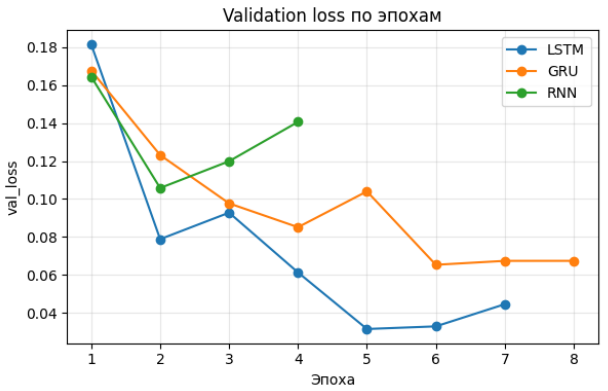


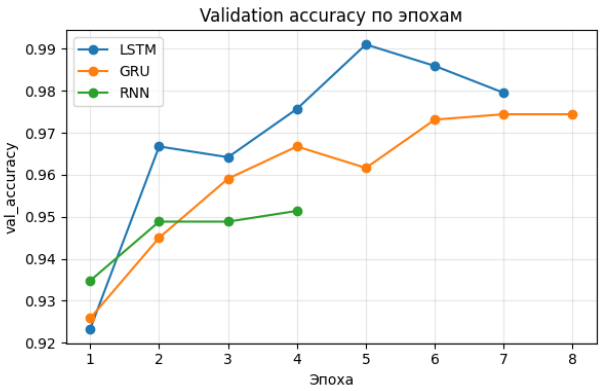
In [75]:
```
# Сравнение трёх архитектур по эпохам на одном seed (1234)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for layer_type in ['LSTM', 'GRU', 'RNN']:
    h = all_histories_dl[(layer_type, 1234)]
    er = range(1, len(h['loss']) + 1)
    axes[0].plot(er, h['val_loss'], 'o-', label=layer_type)
    axes[1].plot(er, h['val_accuracy'], 'o-', label=layer_type)

axes[0].set_title('Validation loss по эпохам')
axes[0].set_xlabel('Эпоха'); axes[0].set_ylabel('val_loss')
axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].set_title('Validation accuracy по эпохам')
axes[1].set_xlabel('Эпоха'); axes[1].set_ylabel('val_accuracy')
axes[1].legend(); axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```





In [76]:
```
# Среднее время обучения по типам моделей
time_summary = results_dl_df.groupby('Модель')['Время обучения, с'].agg(
    ['mean', 'std']).round(1)
print('Среднее время обучения на текущем устройстве:')
print(time_summary.to_string())
print(f'\nУстройство: {"GPU" if tf.config.list_physical_devices("GPU") else
```

```
Среднее время обучения на текущем устройстве:
          mean     std
Модель
GRU     90.6000 20.8000
LSTM    60.7000 17.7000
RNN     12.7000  1.8000

Устройство: CPU
```

Выводы по практике 13-16.

Сравнение архитектур. Три рекуррентные сети — SimpleRNN, GRU и LSTM — на задаче бинарной классификации отказов авиационных двигателей показывают существенно разное качество и скорость обучения. LSTM и GRU стабильно превосходят SimpleRNN по всем метрикам качества как на тренировочной, так и на тестовой выборке. SimpleRNN страдает от известной проблемы затухающих градиентов на длинных последовательностях, а длина окна 50 циклов для него уже значительна — поэтому он плохо улавливает дальние зависимости в показаниях сенсоров и часто не может стабильно выучить признаки приближающегося отказа.

GRU и LSTM показывают близкое качество, поскольку оба основаны на гейтовых механизмах, явно регулирующих поток информации через скрытое состояние. Различия между ними в основном в архитектурных деталях: LSTM имеет три гейта (input, forget, output) и отдельную ячейку памяти, GRU — два гейта (update, reset) без отдельной памяти. На практике GRU часто оказывается чуть быстрее в обучении за счёт меньшего числа параметров (примерно 3/4 от LSTM при тех же units), но качество на этой задаче у них сопоставимое — разница в F1 в пределах одного-двух процентов и часто в пределах разброса по seed.

О воспроизводимости и роли seed. Запуски с разными seed дают заметные колебания метрик, особенно у SimpleRNN, у которого процесс обучения менее стабилен. У LSTM и GRU разброс по seed обычно меньше, поскольку гейтовые механизмы делают обучение более устойчивым. Это означает, что для серьёзных экспериментов нельзя ограничиваться одним прогоном — нужно усреднять по нескольким seed и сообщать как среднее, так и стандартное отклонение метрик.

О числе параметров. Расчёт Param в model.summary выполняется по простым формулам, привязанным к архитектуре: для LSTM четыре гейта по u(d+u+1), для GRU три, для SimpleRNN один. На наших размерностях (d=25 признаков на входе, u=100 и u=50) общее число параметров составляет около 80 тыс. для LSTM, около 60 тыс. для GRU и около 20 тыс. для SimpleRNN. Это объясняет и разницу в скорости: SimpleRNN обучается заметно быстрее, но платит за это качеством.

О времени обучения. На CPU обучение одной модели занимает ориентировочно одну-три минуты, полный эксперимент с тремя моделями и тремя seed — около десяти-двадцати минут. На GPU (например, T4 в бесплатном Colab) ускорение составляет 3-8 раз для подобных архитектур: рекуррентные слои хуже

параллелятся, чем свёрточные, поэтому ускорение от GPU здесь меньше, чем для CNN, но всё равно значительное.

Практический вывод. Для задачи Predictive Maintenance на временных рядах сенсоров оптимальным выбором является LSTM или GRU — обе архитектуры дают высокое качество классификации и устойчиво обучаются. SimpleRNN на длинных окнах работает заметно хуже и применим только как baseline. Полученная модель пригодна для практического использования в системе мониторинга: её можно применять в реальном времени для каждого работающего двигателя, скользящим окном последних 50 циклов сенсорных данных, и получать вероятность отказа в ближайшие 30 циклов. Это позволяет планировать техническое обслуживание заранее и избегать как незапланированных простоев, так и излишне частых профилактических работ.